



第12章 查询执行

清华大学数据库组 李国良

liguoliang@tsinghua.edu.cn

<http://dbgroup.cs.tsinghua.edu.cn/ligl>

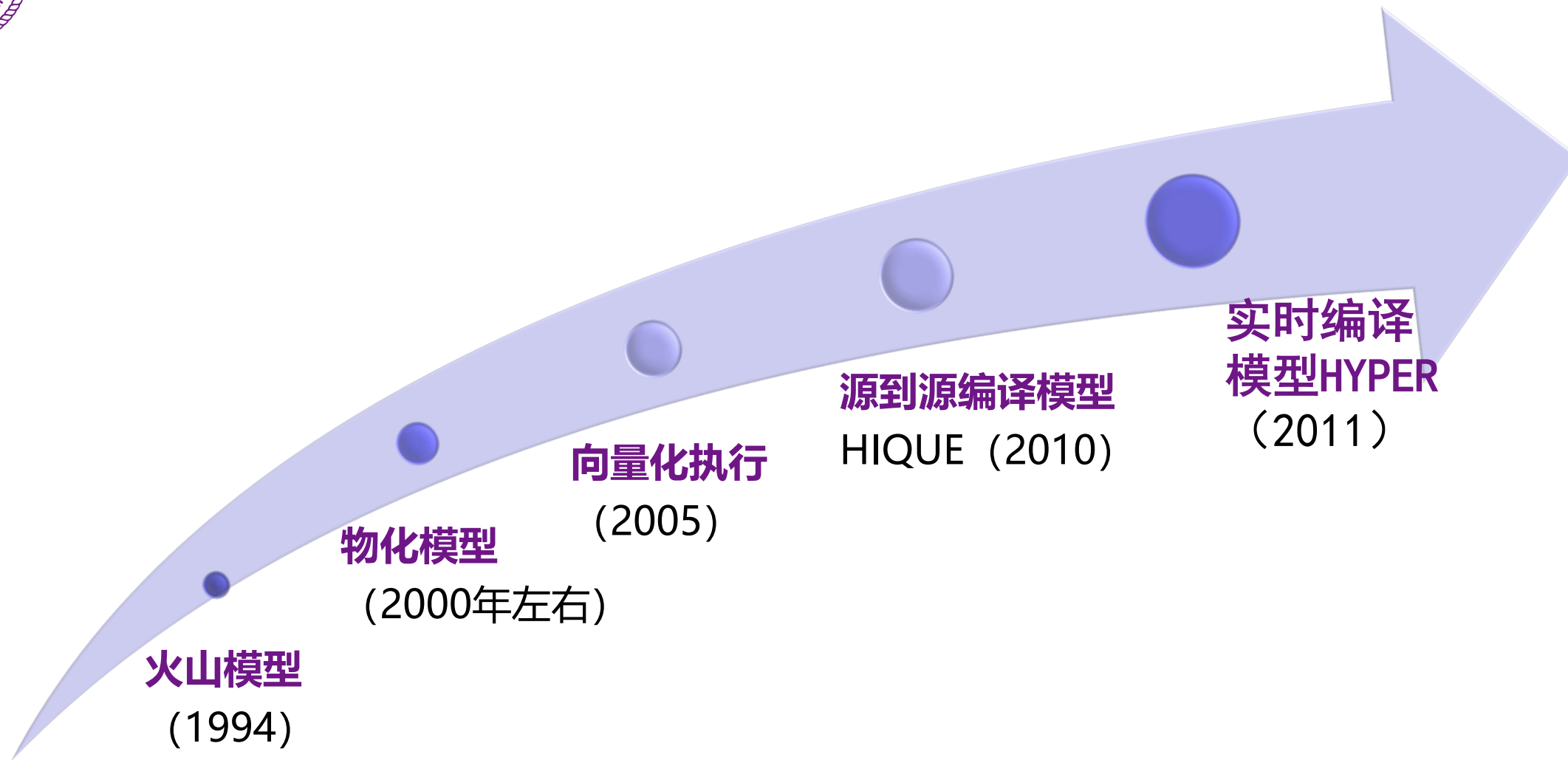


查询执行概述

- 查询执行模块的输入是物理执行计划，而输出则是可执行的编程代码，甚至是机器码。
- 查询执行模型的优化目标：以更短的时间来完成精确的查询。
- 对于这一目标有三种可行的方法：
 - (1) 通过合理规划减少总的指令数量。
 - (2) 减少循环数量，尽量在更少的循环数量中做更多的事。
 - (3) 并行处理查询：数据并行、计算并行



查询执行发展过程





目录

01

执行器概述

02

拉取式模型

03

推送式模型

04

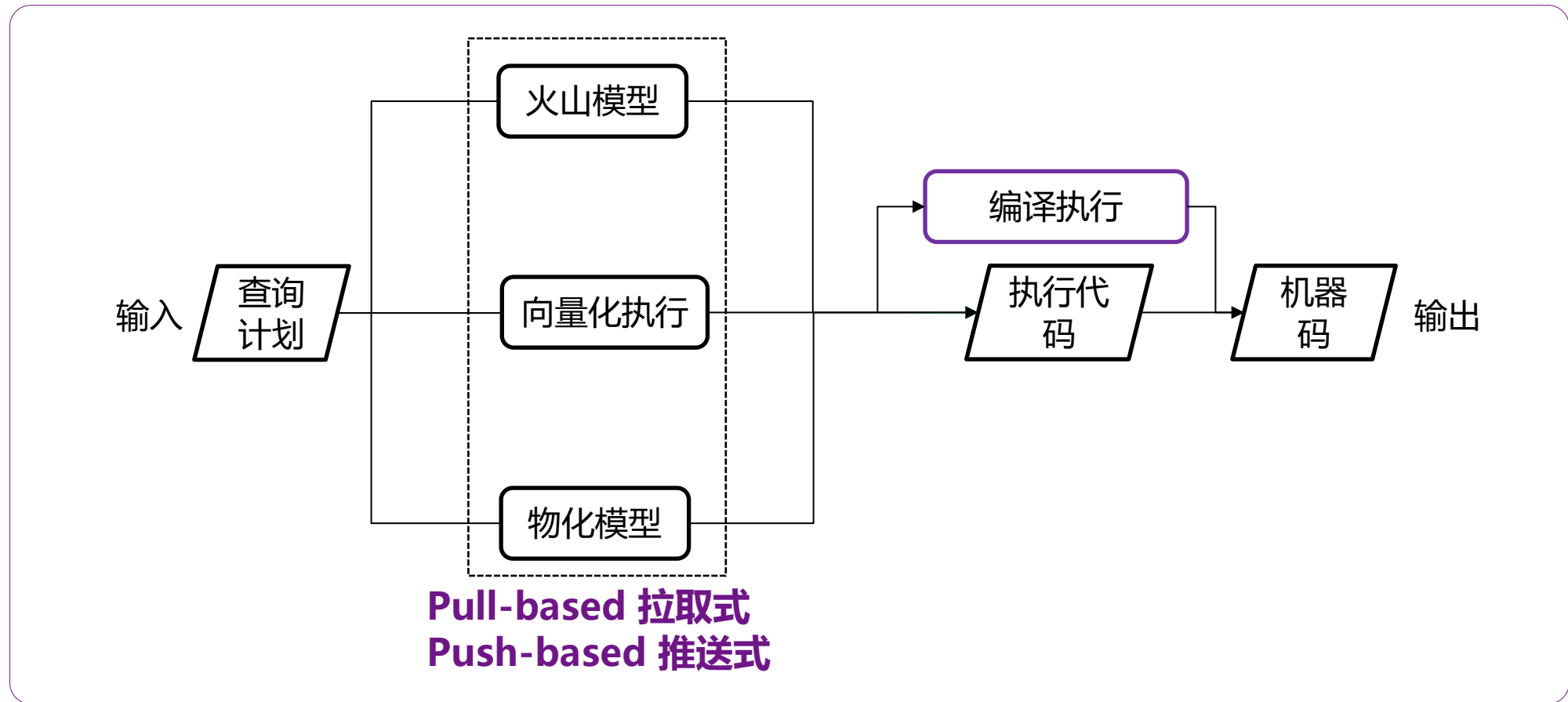
并行执行

05

编译执行



查询执行框架





拉取式模型 Pull-based

➤ Open()

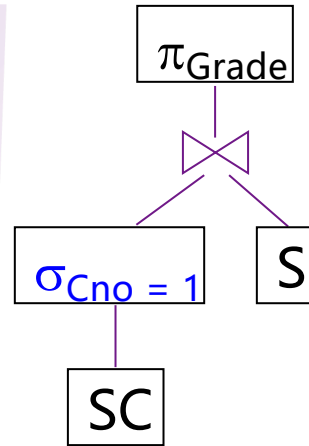
- 初始化
- 申请空间和设置连接、选择条件的参数

➤ getNext()

- 获取下一条/一批记录
- 验证是否满足条件，若满足返回父节点；否则中止

➤ Close()

- 清理



```
Open() { /* initialize variables */  
    b = first block of SC; // malloc a buffer  
    t = first tuple in block b;  
}
```

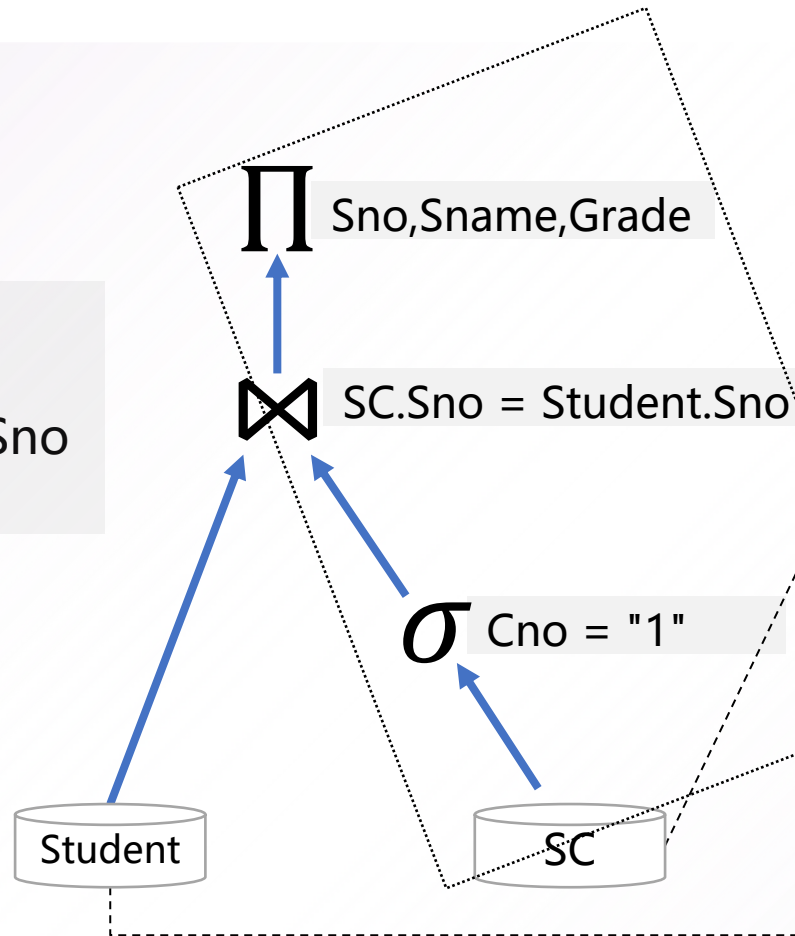
```
getNext() {  
    IF (there is no next tuple in block b):  
        set b to next block;  
        IF(there is no next block): /* no more tuples */  
            RETURN NULL;  
        ELSE: t = first tuple in b;  
        next-first = t; /* return current tuple */  
        set t to next tuple in block b;  
        RETURN next-first;  
}
```

```
Close() { /* nothing to be done */  
    free the buffer of b;  
}
```



推送式模型 Push-based

```
SELECT Sno, Sname, Grade  
FROM SC, Student  
WHERE SC.Sno = Student.Sno  
AND Cno = "1"
```

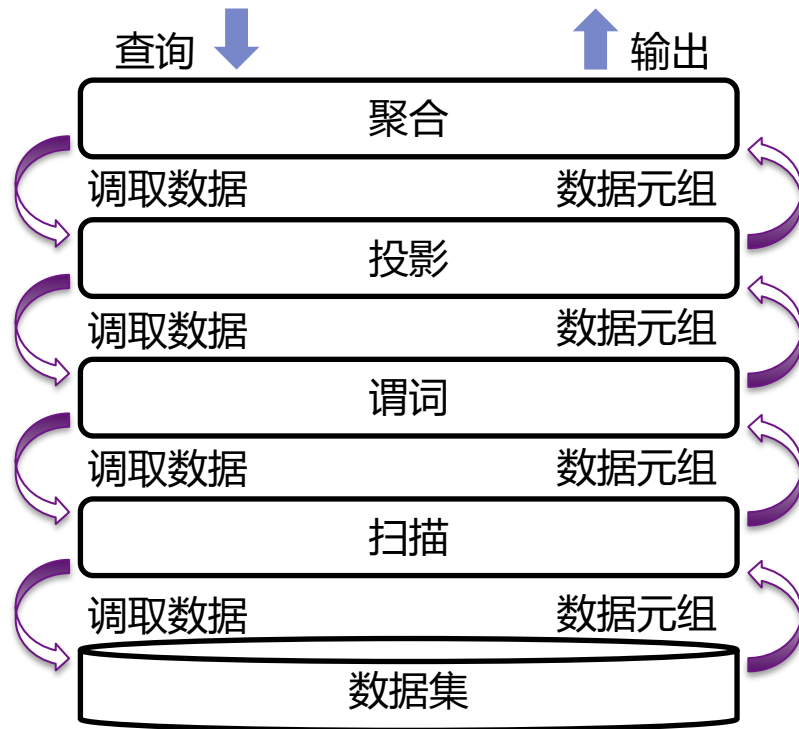


For t_2 in SC:
if evalPred(t_2): //cno=1
if probe(t_2) in HT: // t_1 in HT[t_2]
emit($t_1 \bowtie t_2$)

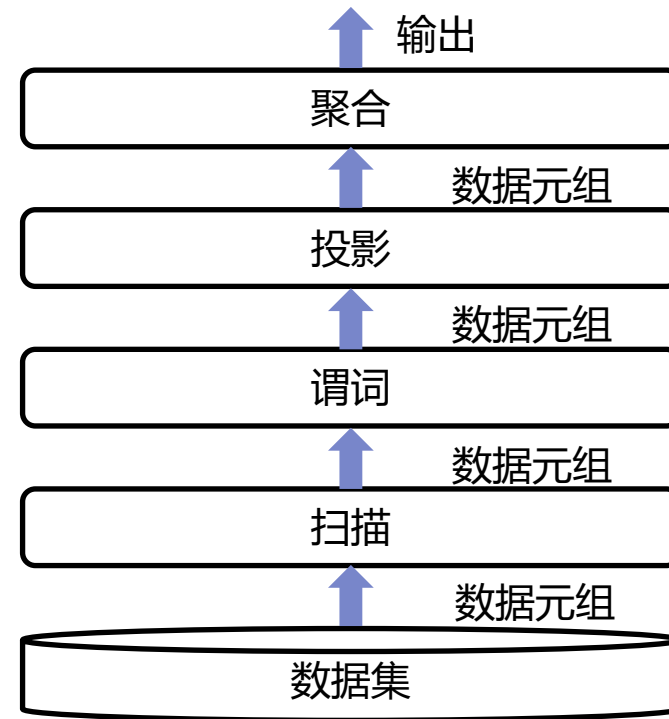
for t_1 in Student:
HT=BuildHashTable(t_1)



拉取式模型与推送式模型



拉取式模型



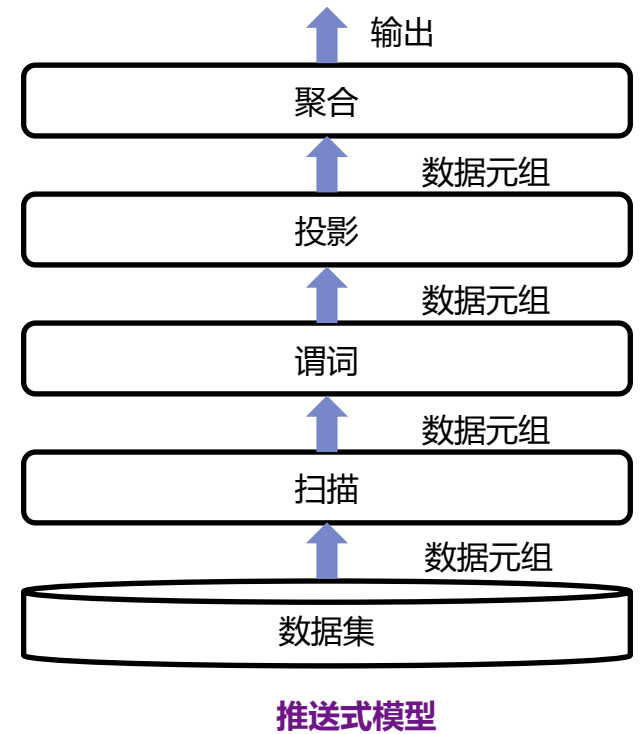
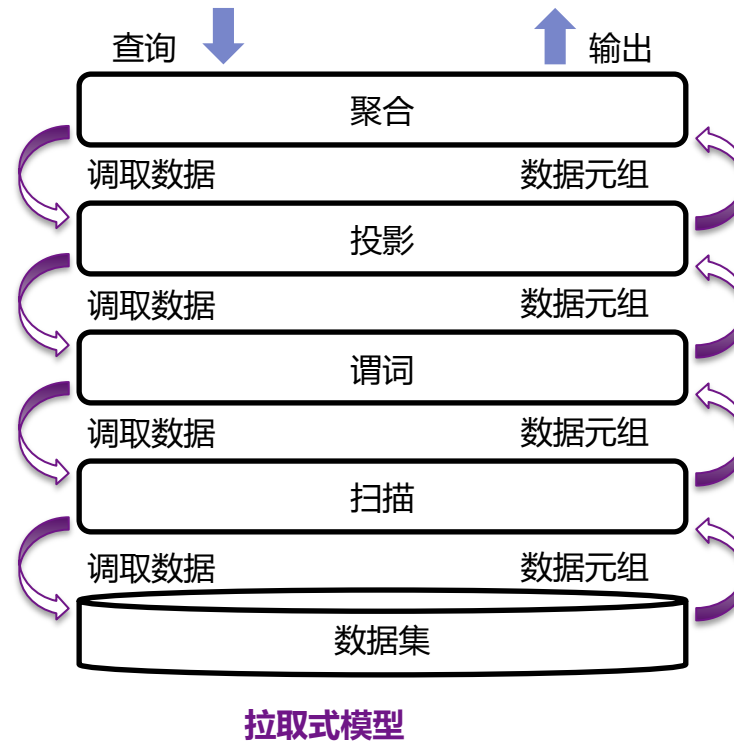
推送式模型



拉取式模型与推送式模型



	拉取式	推送式
开始节点	根节点	叶子节点
执行方向	自上而下	自下而上
核心	以算子为核心	以数据为核心
执行方式	通过函数从子节点“拉取数据”	将数据“推送”给父节点
优势	省内存、易实现	效率高、易并行
劣势	函数调用多	limit效率低，内存消耗高





目录

01

执行器概述

02

拉取式模型

03

推送式模型

04

并行执行

05

编译执行



拉取式：火山 vs 物化 vs 向量化模型



➤ 火山

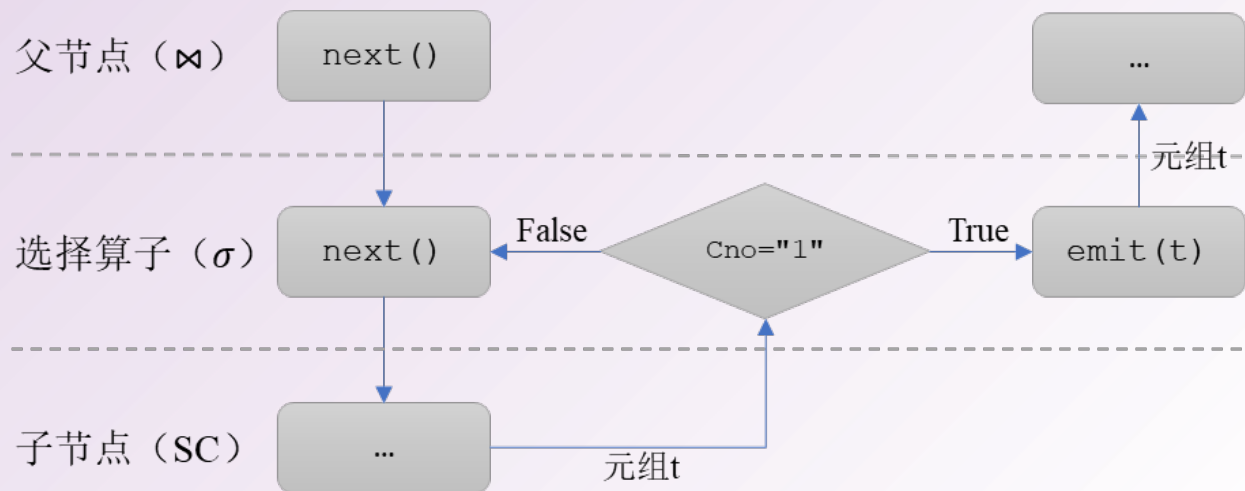
- 一次一元组

➤ 物化

- 一次所有元组

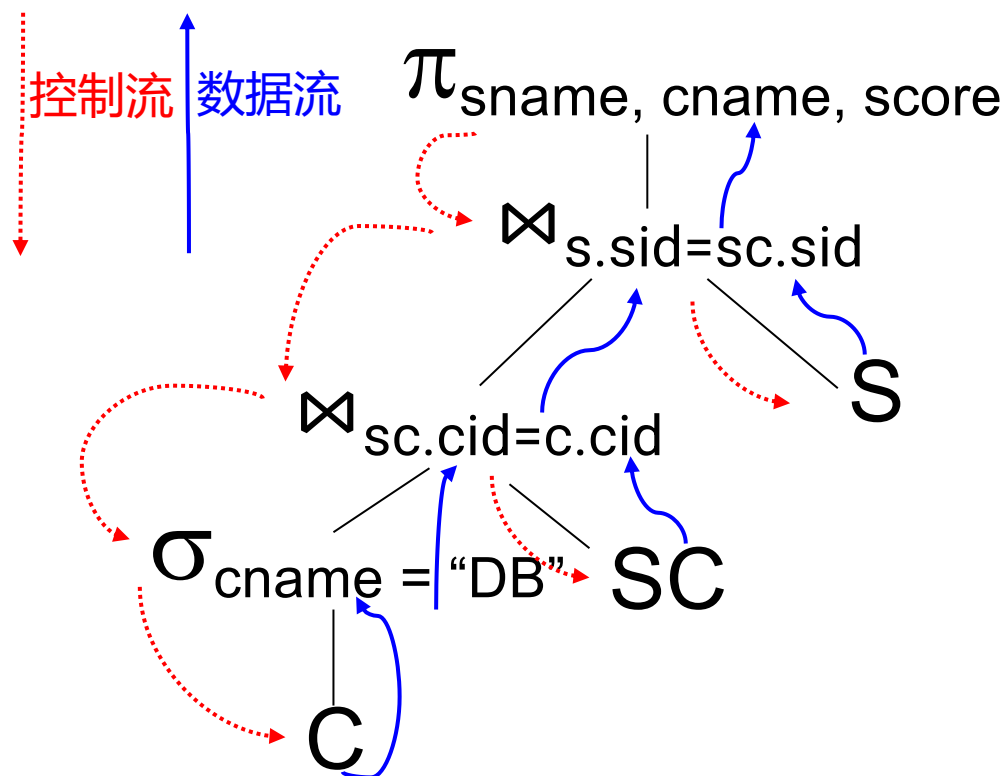
➤ 向量化

- 一次一批元组



函数调用:

- `Open()`
- `GetNext()`
- `Close()`





火山模型

- 火山模型 (volcano model) 又被称为迭代模型 (iterator model) 或流水线模型 (pipeline model) ;
- 是一种以算子为核心, 自上而下的拉取式的执行模型;
- 所有的算子都被视为独立执行的程序, 并为它们提供了一个共同的应用程序接口, 以方便拓展应用;
- 查询自上而下地通过next()函数递归调取数据元组, 而元组流向则是自下而上。

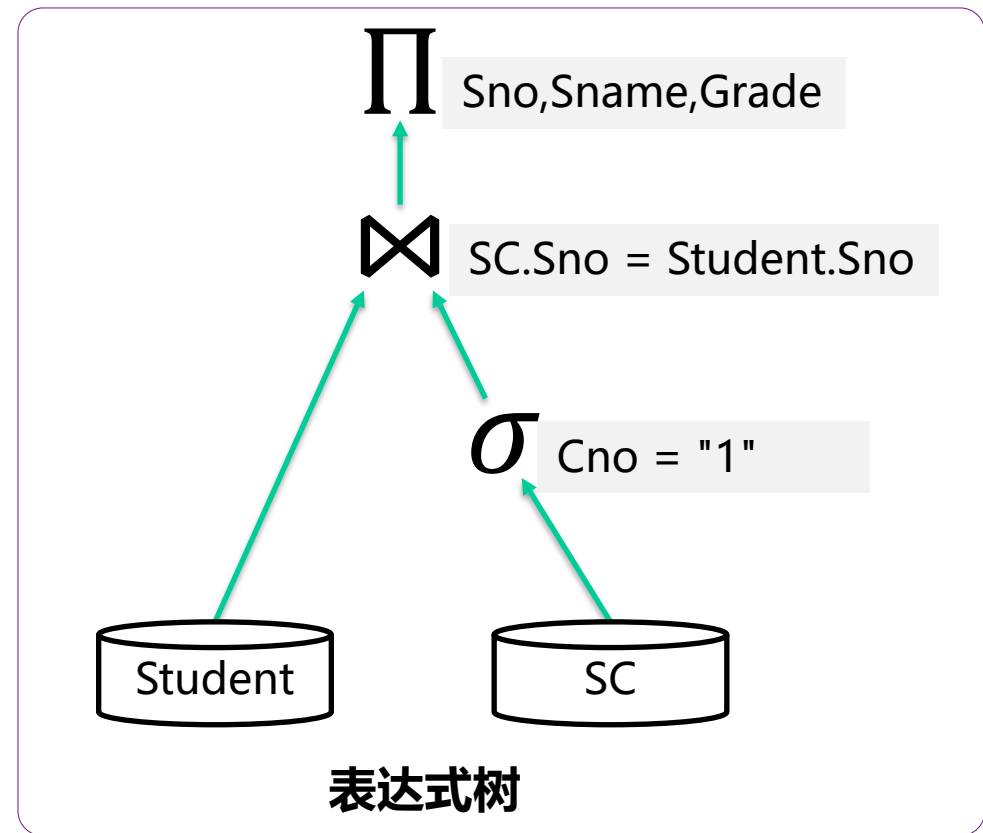


火山模型示例



- 查询获取“选修了1号课程的学生学号、姓名及成绩”

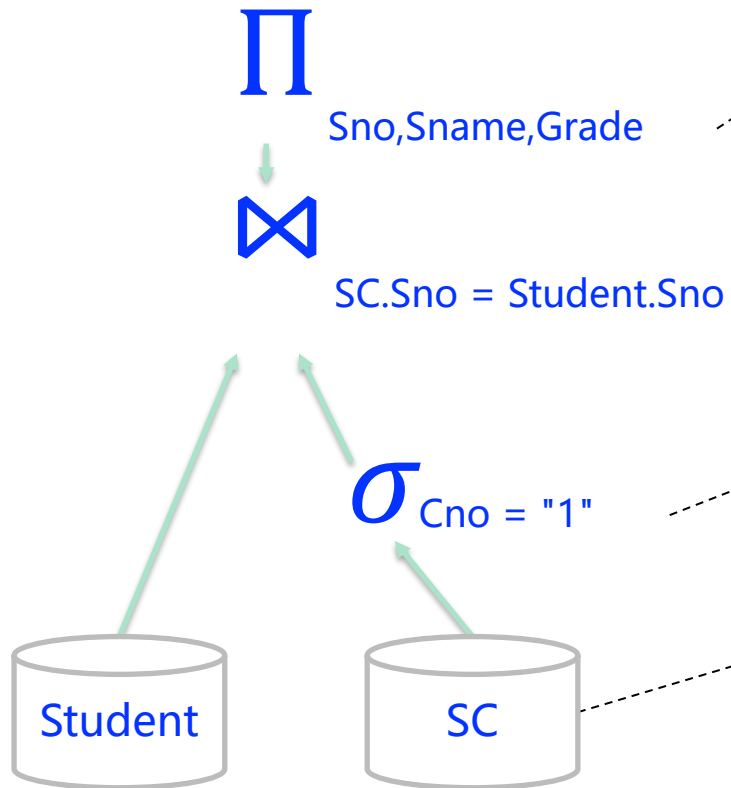
```
SELECT Sno, Sname, Grade  
FROM SC, Student  
WHERE SC.Sno = Student.Sno  
AND Cno = "1"
```





火山模型执行流程

```
SELECT Sno, Sname, Grade
FROM SC, Student
WHERE SC.Sno = Student.Sno
AND Cno = "1"
```



for t in child.next():
 emit(projection(t))

for t_1 in left.next():
 BuildHashTable(t_1)
for t_2 in right.next():
 if probe(t_2): emit($t_1 \bowtie t_2$)

for t in child.next():
 if evalPred(t): emit(t)

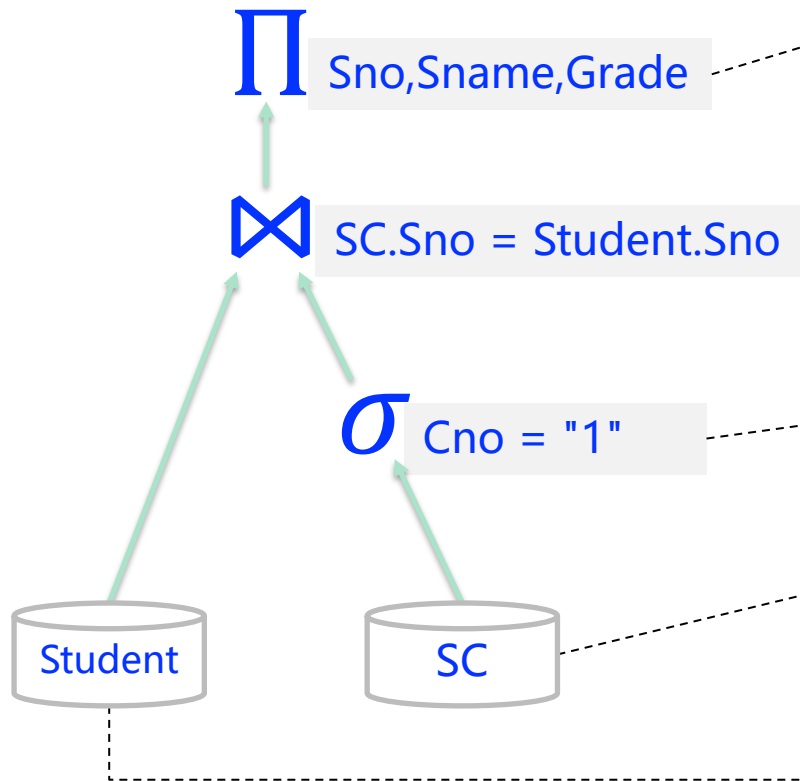
for t in SC:
 emit(t)

for t in Student:
 emit(t)



火山模型执行流程

```
SELECT Sno, Sname, Grade
FROM SC, Student
WHERE SC.Sno = Student.Sno
AND Cno = "1"
```



```
for  $t$  in child.next():
    emit(projection( $t$ ))
```

```
for  $t_1$  in left.next():
    BuildHashTable( $t_1$ )
for  $t_2$  in right.next():
    if probe( $t_2$ ): emit( $t_1 \bowtie t_2$ )
```

```
for  $t$  in child.next():
    if evalPred( $t$ ): emit( $t$ )
```

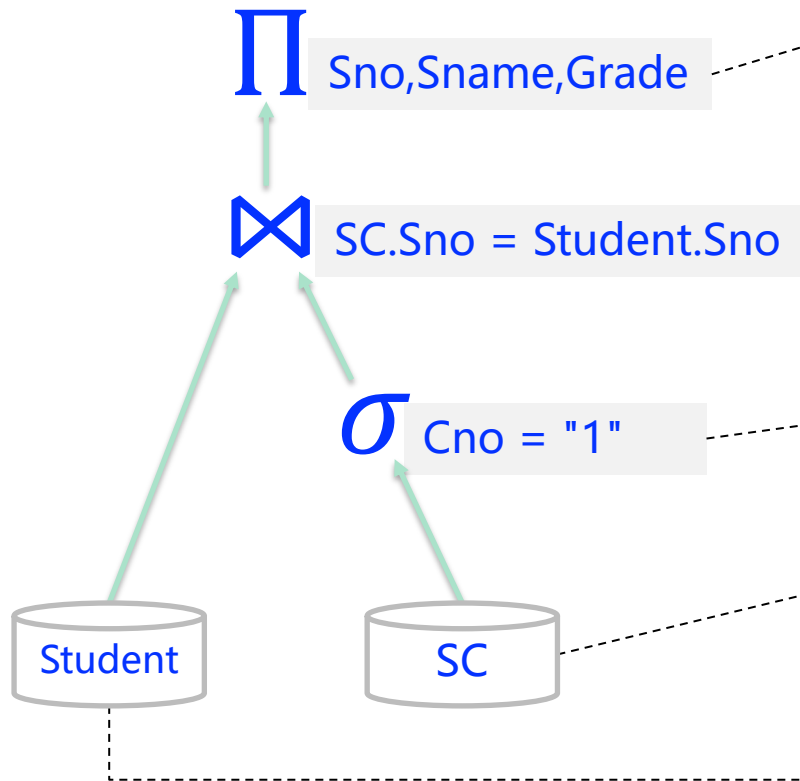
```
for  $t$  in SC:
    emit( $t$ )
```

```
for  $t$  in Student:
    emit( $t$ )
```



火山模型执行流程

```
SELECT Sno, Sname, Grade
FROM SC, Student
WHERE SC.Sno = Student.Sno
AND Cno = "1"
```



```
for  $t$  in child.next():
    emit(projection( $t$ ))
```

```
for  $t_1$  in left.next():
    BuildHashTable( $t_1$ )
for  $t_2$  in right.next():
    if probe( $t_2$ ): emit( $t_1 \bowtie t_2$ )
```

```
for  $t$  in child.next():
    if evalPred( $t$ ): emit( $t$ )
```

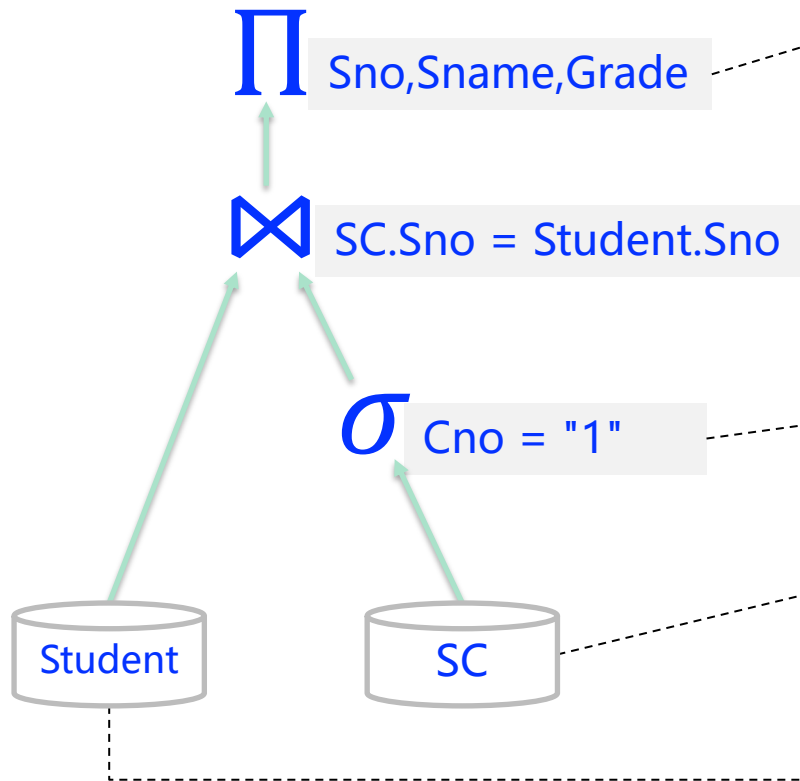
```
for  $t$  in SC:
    emit( $t$ )
```

```
for  $t$  in Student:
    emit( $t$ )
```



火山模型执行流程

```
SELECT Sno, Sname, Grade
FROM SC, Student
WHERE SC.Sno = Student.Sno
AND Cno = "1"
```



```
for t in child.next():
    emit(projection(t))
```

```
for t1 in left.next():
    BuildHashTable(t1)
for t2 in right.next():
    if probe(t2): emit(t1 ⋈ t2)
```

```
for t in child.next():
    if evalPred(t): emit(t)
```

```
for t in SC:
    emit(t)
```

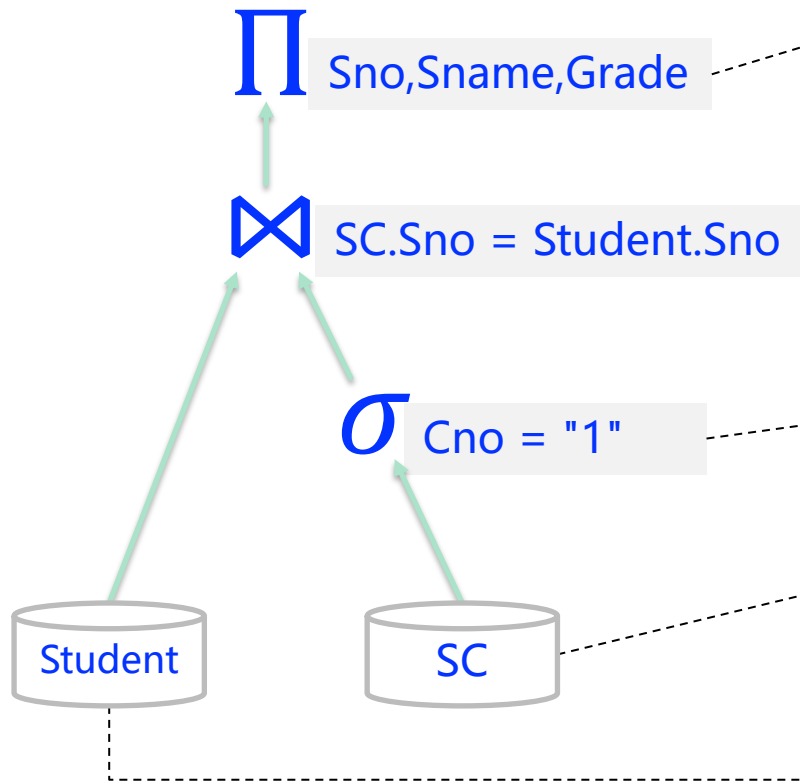
```
for t in Student:
    emit(t)
```

每次返回一条元组



火山模型执行流程

```
SELECT Sno, Sname, Grade
FROM SC, Student
WHERE SC.Sno = Student.Sno
AND Cno = "1"
```



```
for t in child.next():
    emit(projection(t))
```

```
for t1 in left.next():
    BuildHashTable(t1)
for t2 in right.next():
    if probe(t2): emit(t1  $\bowtie$  t2)
```

```
for t in child.next():
    if evalPred(t): emit(t)
```

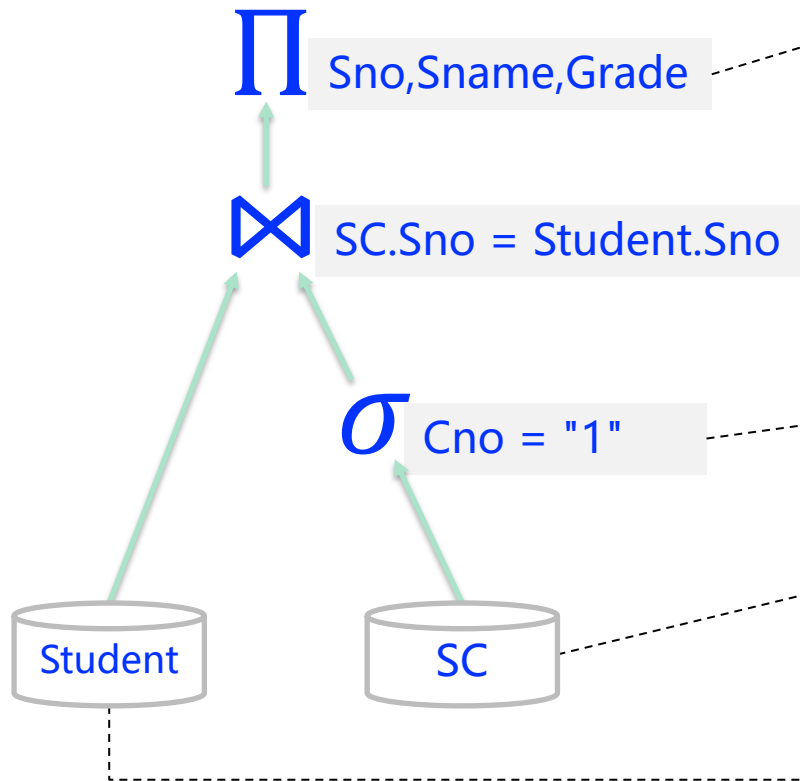
```
for t in SC:
    emit(t)
```

```
for t in Student:
    emit(t)
```




火山模型执行流程

```
SELECT Sno, Sname, Grade
FROM SC, Student
WHERE SC.Sno = Student.Sno
AND Cno = "1"
```



```
for t in child.next():
    emit(projection(t))
```

```
for t1 in left.next():
    BuildHashTable(t1)
for t2 in right.next():
    if probe(t2): emit(t1 ⋈ t2)
```

```
for t in child.next():
    if evalPred(t): emit(t)
```

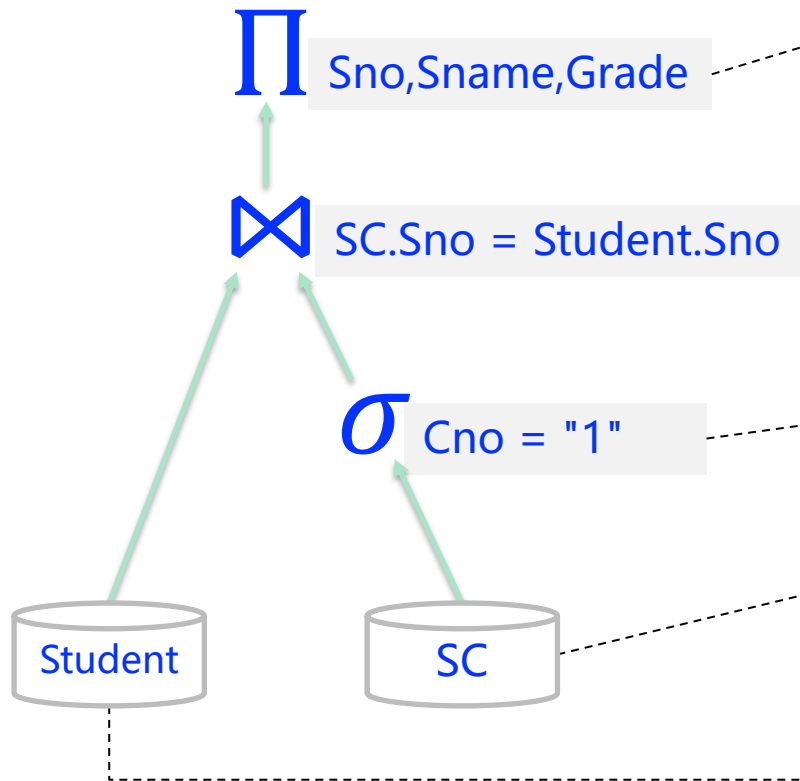
```
for t in SC:
    emit(t)
```

```
for t in Student:
    emit(t)
```



火山模型执行流程

```
SELECT Sno, Sname, Grade
FROM SC, Student
WHERE SC.Sno = Student.Sno
AND Cno = "1"
```



```
for t in child.next():
    emit(projection(t))
```

```
for t1 in left.next():
    BuildHashTable(t1)
for t2 in right.next():
    if probe(t2): emit(t1 ⋈ t2)
```

```
for t in child.next():
    if evalPred(t): emit(t)
```

```
for t in SC:
    emit(t)
```

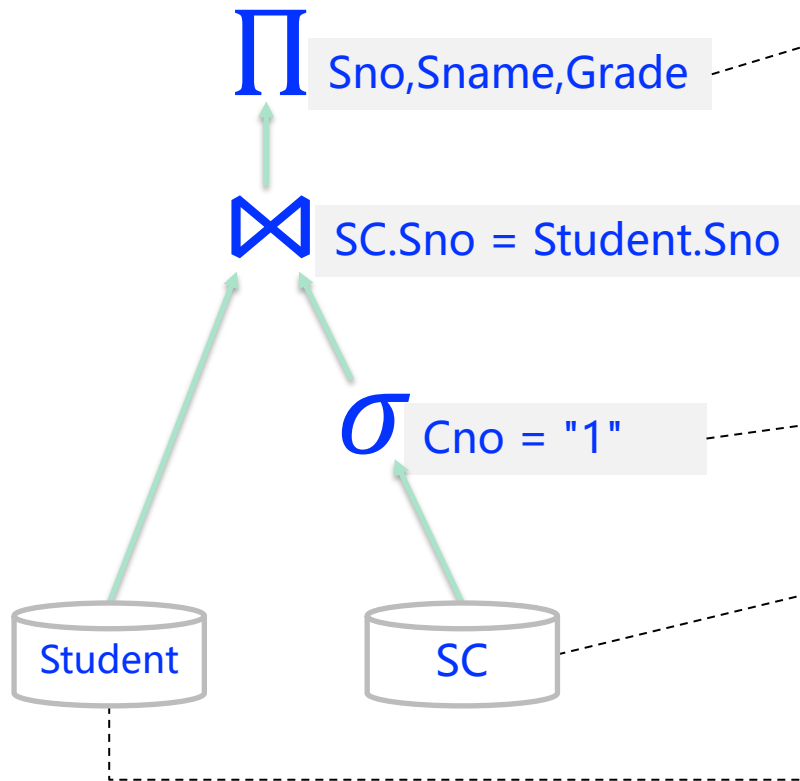
```
for t in Student:
    emit(t)
```

每次返回一条元组



火山模型执行流程

```
SELECT Sno, Sname, Grade
FROM SC, Student
WHERE SC.Sno = Student.Sno
AND Cno = "1"
```



```
for  $t$  in child.next():
    emit(projection( $t$ ))
```

```
for  $t_1$  in left.next():
    BuildHashTable( $t_1$ )
for  $t_2$  in right.next():
    if probe( $t_2$ ): emit( $t_1 \bowtie t_2$ )
```

```
for  $t$  in child.next():
    if evalPred( $t$ ): emit( $t$ )
```

```
for  $t$  in SC:
    emit( $t$ )
```

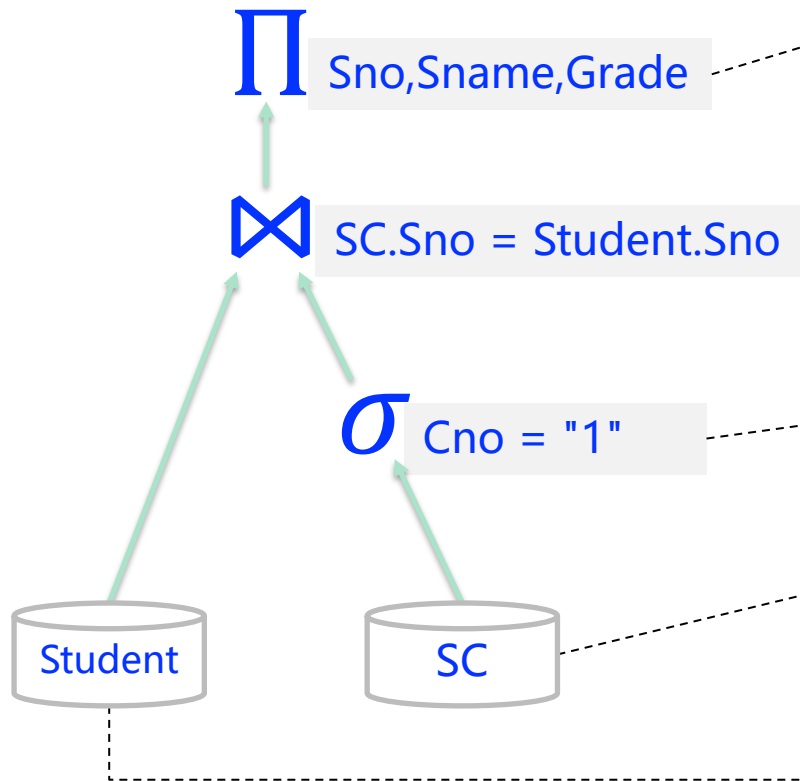
```
for  $t$  in Student:
    emit( $t$ )
```

每次返回一条元组



火山模型执行流程

```
SELECT Sno, Sname, Grade
FROM SC, Student
WHERE SC.Sno = Student.Sno
AND Cno = "1"
```



```
for t in child.next():
    emit(projection(t))
```

```
for t1 in left.next():
    BuildHashTable(t1)
for t2 in right.next():
    if probe(t2): emit(t1 ⋈ t2)
```

```
for t in child.next():
    if evalPred(t): emit(t)
```

```
for t in SC:
    emit(t)
```

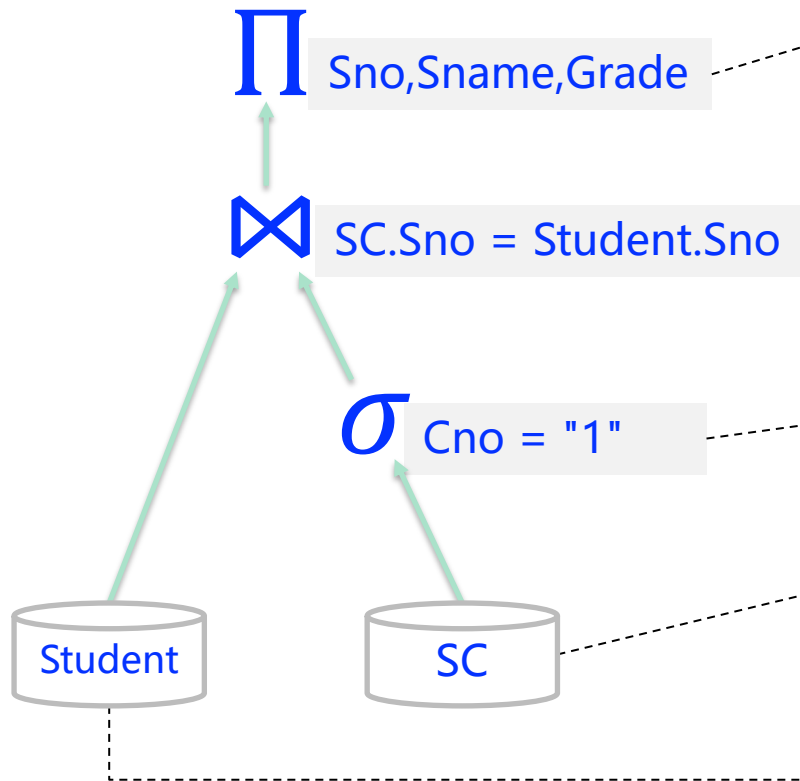
```
for t in Student:
    emit(t)
```

每次返回一条元组



火山模型执行流程

```
SELECT Sno, Sname, Grade
FROM SC, Student
WHERE SC.Sno = Student.Sno
AND Cno = "1"
```



```
for  $t$  in child.next():
    emit(projection( $t$ ))
```

```
for  $t_1$  in left.next():
    BuildHashTable( $t_1$ )
for  $t_2$  in right.next():
    if probe( $t_2$ ): emit( $t_1 \bowtie t_2$ )
```

```
for  $t$  in child.next():
    if evalPred( $t$ ): emit( $t$ )
```

```
for  $t$  in SC:
    emit( $t$ )
```

```
for  $t$  in Student:
    emit( $t$ )
```



物化模型

- 物化模型 (Materialization Model) 以数据为中心，自下而上地推送数据；
- 算子会将所有输入处理好后，将处理完的所有结果一次性输出；
- 输出既可以是完整的元组，也可以只包含其中部分的列。

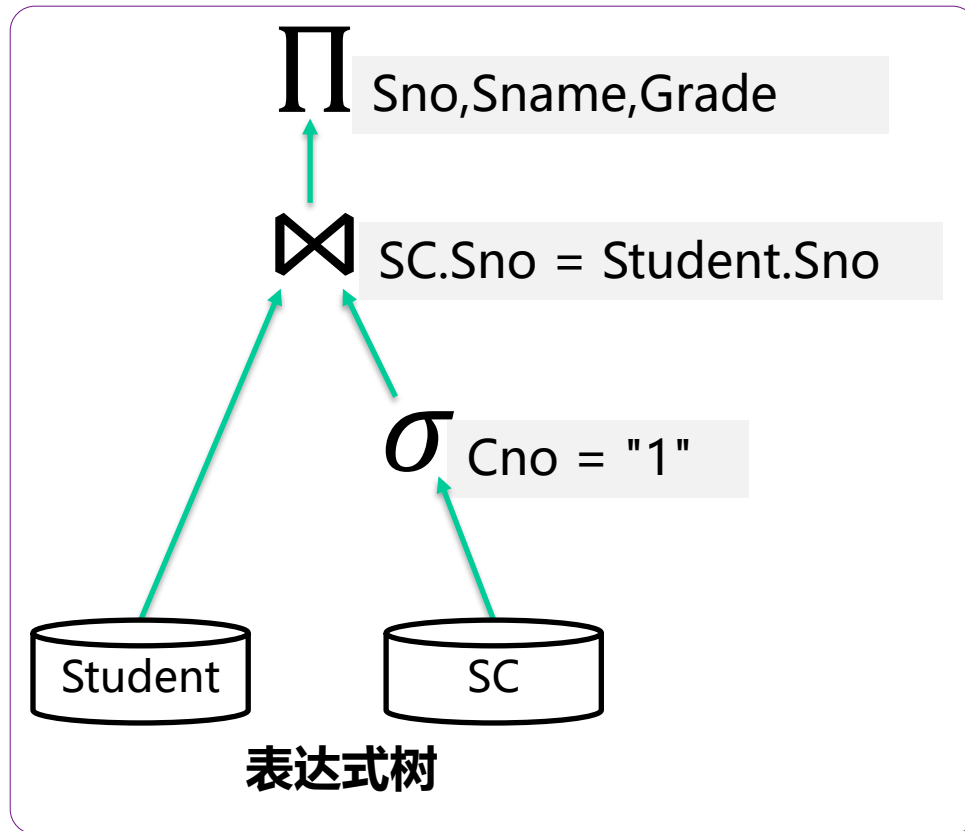


物化模型示例



- 查询获取“选修了1号课程的学生学号、姓名及成绩”

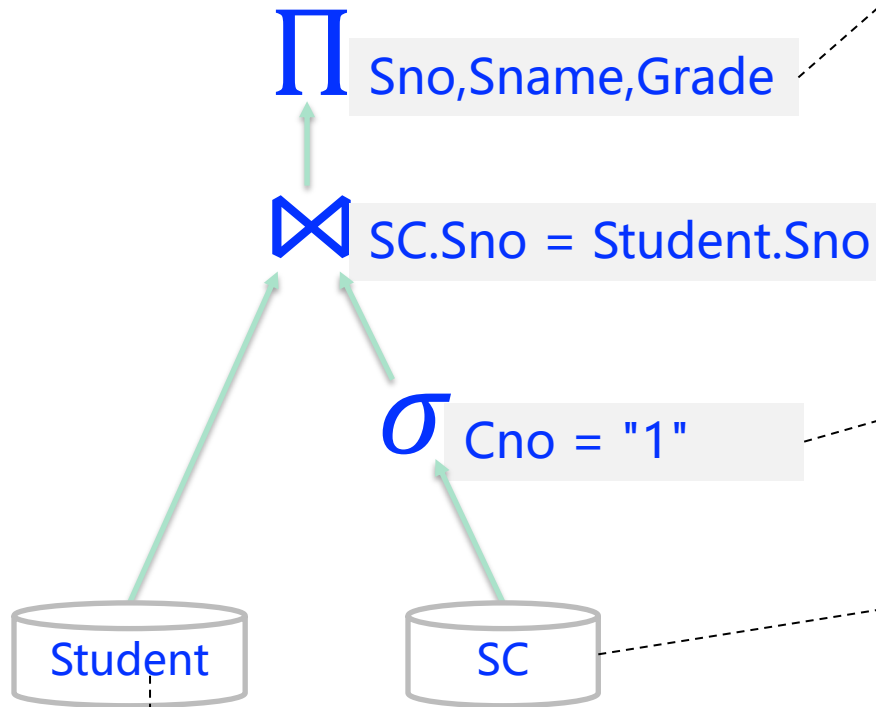
```
SELECT Sno, Sname, Grade
FROM SC, Student
WHERE SC.Sno = Student.Sno
AND Cno = "1"
```





物化模型执行流程

```
SELECT Sno, Sname, Grade
FROM SC, Student
WHERE SC.Sno = Student.Sno
AND Cno = "1"
```



```
out = []
for t in child.output():
    out.add(projection(t))
return out
```

```
out = []
for t1 in left.output():
    BuildHashTable(t1)
for t2 in right.output():
    if probe(t2): out.add(t1 ⋈ t2)
return out
```

```
out = []
for t in child.output():
    if evalPred(t): out.add(t)
return out
```

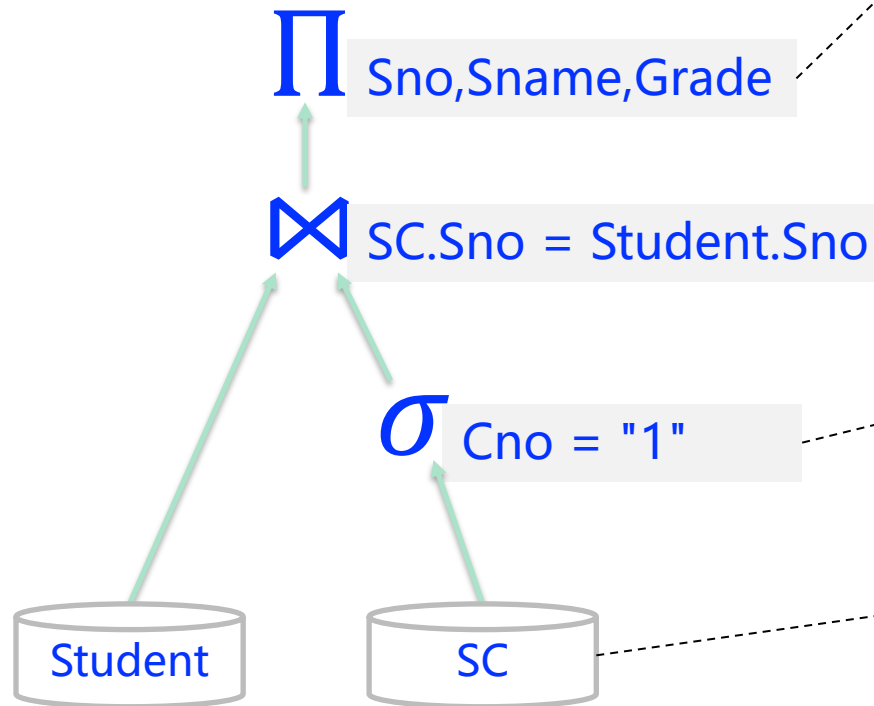
```
out = []
for t in SC: out.add(t)
return out
```

```
out = []
for t in Student: out.add(t)
return out
```




物化模型执行流程

```
SELECT Sno, Sname, Grade
FROM SC, Student
WHERE SC.Sno = Student.Sno
AND Cno = "1"
```



```
out = []
for t in child.output():
    out.add(projection(t))
return out
```

```
out = []
for t1 in left.output():
    BuildHashTable(t1)
for t2 in right.output():
    if probe(t2): out.add(t1 ⋈ t2)
return out
```

```
out = []
for t in child.output():
    if evalPred(t): out.add(t)
return out
```

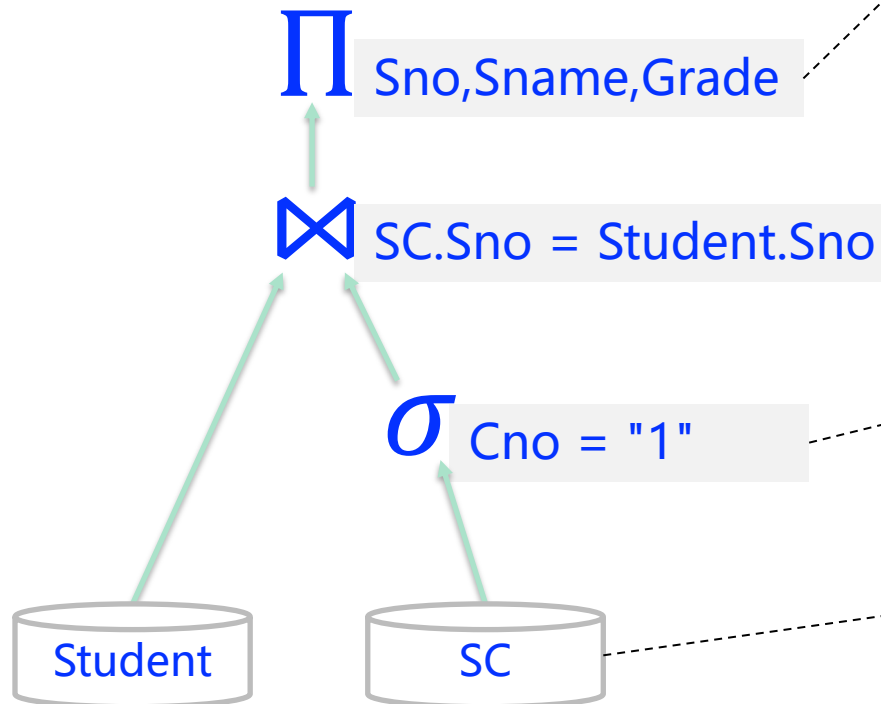
```
out = []
for t in SC: out.add(t)
return out
```

```
out = []
for t in Student: out.add(t)
return out
```



物化模型执行流程

```
SELECT Sno, Sname, Grade
FROM SC, Student
WHERE SC.Sno = Student.Sno
AND Cno = "1"
```



```
out = []
for t in child.output():
    out.add(projection(t))
return out
```

```
out = []
for t1 in left.output():
    BuildHashTable(t1)
for t2 in right.output():
    if probe(t2): out.add(t1 ⋈ t2)
return out
```

```
out = []
for t in child.output():
    if evalPred(t): out.add(t)
return out
```

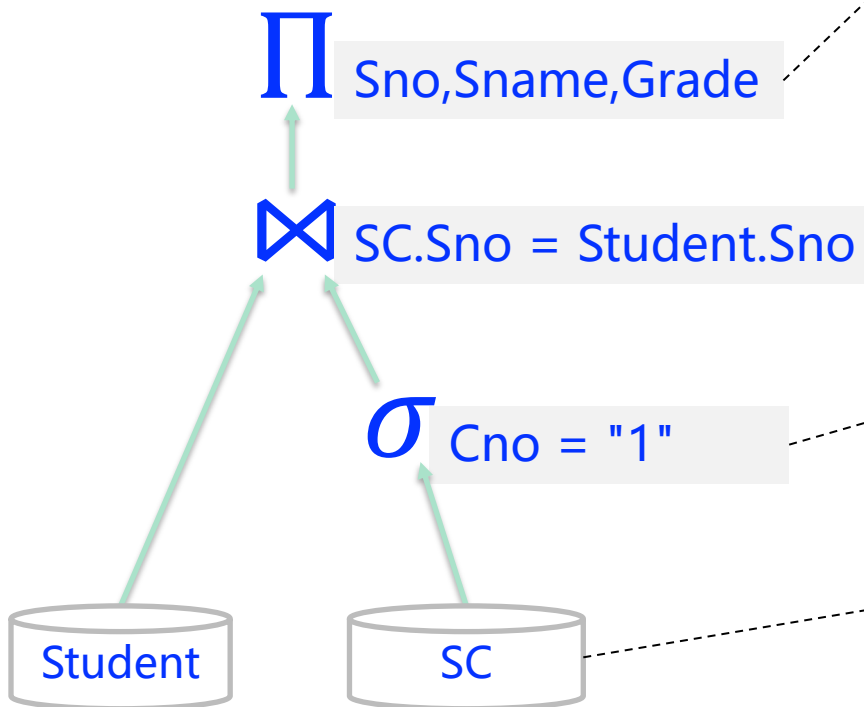
```
out = []
for t in SC: out.add(t)
return out
```

```
out = []
for t in Student: out.add(t)
return out
```



物化模型执行流程

```
SELECT Sno, Sname, Grade
FROM SC, Student
WHERE SC.Sno = Student.Sno
AND Cno = "1"
```



```
out = []
for t in child.output():
    out.add(projection(t))
return out
```

```
out = []
for t1 in left.output():
    BuildHashTable(t1)
for t2 in right.output():
    if probe(t2): out.add(t1 ⋈ t2)
return out
```

```
out = []
for t in child.output():
    if evalPred(t): out.add(t)
return out
```

```
out = []
for t in SC: out.add(t)
return out
```

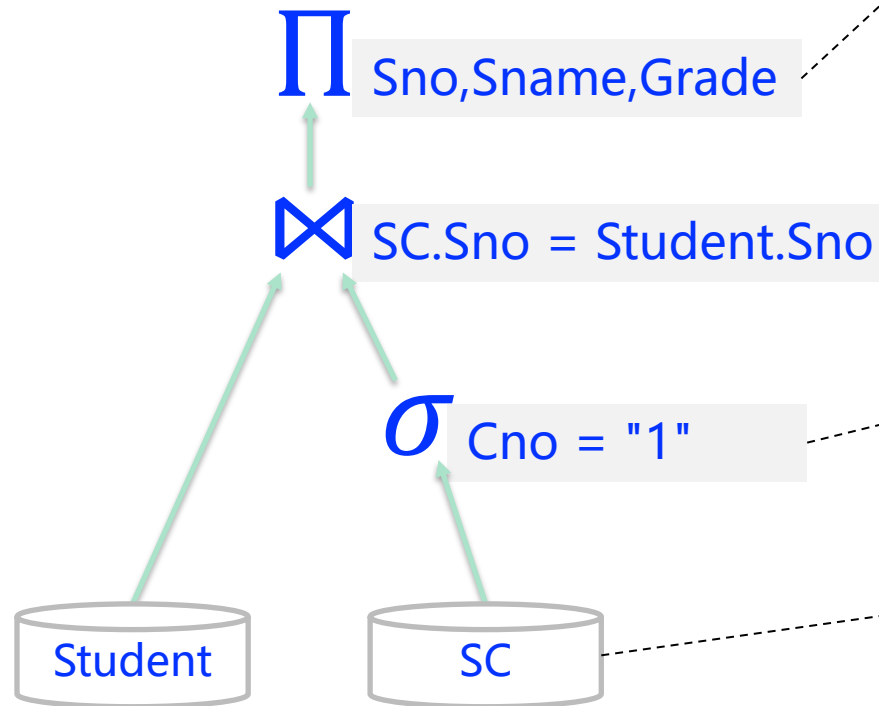
```
out = []
for t in Student: out.add(t)
return out
```

返回全部元组



物化模型执行流程

```
SELECT Sno, Sname, Grade
FROM SC, Student
WHERE SC.Sno = Student.Sno
AND Cno = "1"
```



```
out = []
for  $t$  in child.output():
    out.add(projection( $t$ ))
return out
```

```
out = []
for  $t_1$  in left.output():
    BuildHashTable( $t_1$ )
for  $t_2$  in right.output():
    if probe( $t_2$ ): out.add( $t_1 \bowtie t_2$ )
return out
```

```
out = []
for  $t$  in child.output():
    if evalPred( $t$ ): out.add( $t$ )
return out
```

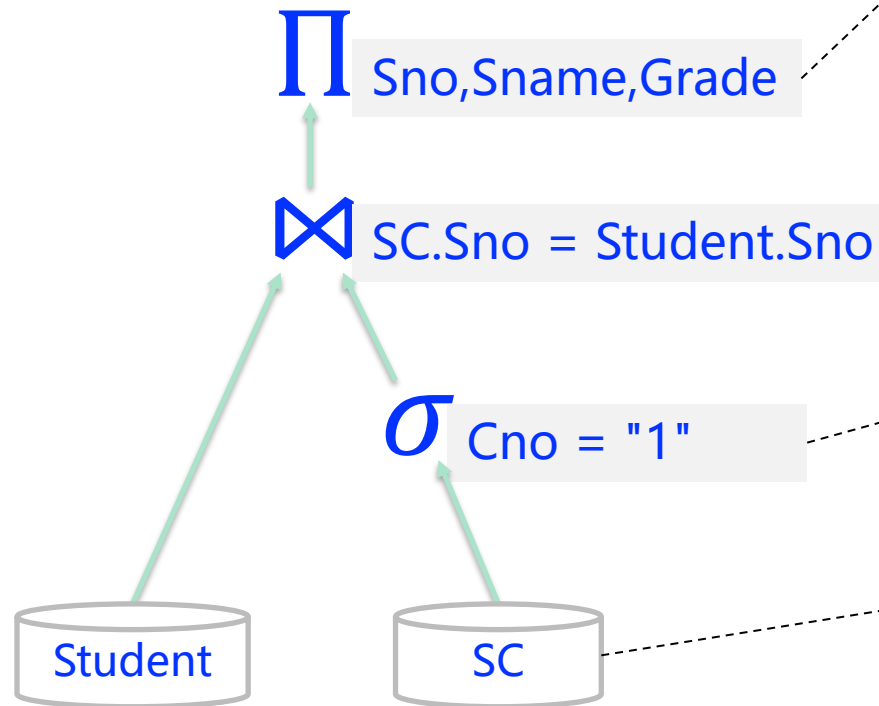
```
out = []
for  $t$  in SC: out.add( $t$ )
return out
```

```
out = []
for  $t$  in Student: out.add( $t$ )
return out
```



物化模型执行流程

```
SELECT Sno, Sname, Grade
FROM SC, Student
WHERE SC.Sno = Student.Sno
AND Cno = "1"
```



```
out = []
for t in child.output():
    out.add(projection(t))
return out
```

```
out = []
for t1 in left.output():
    BuildHashTable(t1)
for t2 in right.output():
    if probe(t2): out.add(t1 ⋈ t2)
return out
```

```
out = []
for t in child.output():
    if evalPred(t): out.add(t)
return out
```

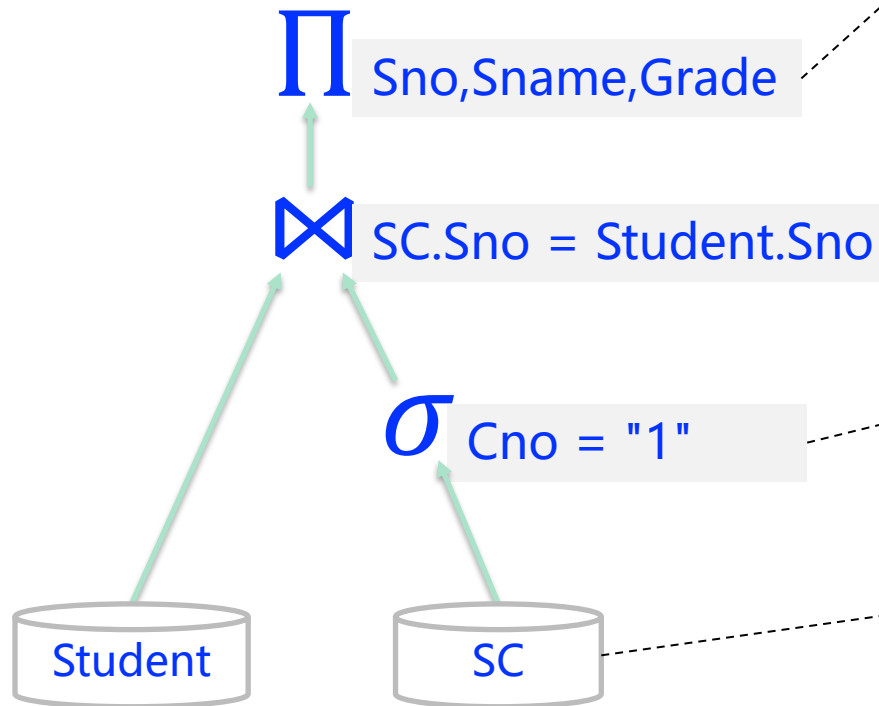
```
out = []
for t in SC: out.add(t)
return out
```

```
out = []
for t in Student: out.add(t)
return out
```



物化模型执行流程

```
SELECT Sno, Sname, Grade
FROM SC, Student
WHERE SC.Sno = Student.Sno
AND Cno = "1"
```



```
out = []
for t in child.output():
    out.add(projection(t))
return out
```

```
out = []
for t1 in left.output():
    BuildHashTable(t1)
for t2 in right.output():
    if probe(t2): out.add(t1 ⋈ t2)
return out
```

```
out = []
for t in child.output():
    if evalPred(t): out.add(t)
return out
```

```
out = []
for t in SC: out.add(t)
return out
```

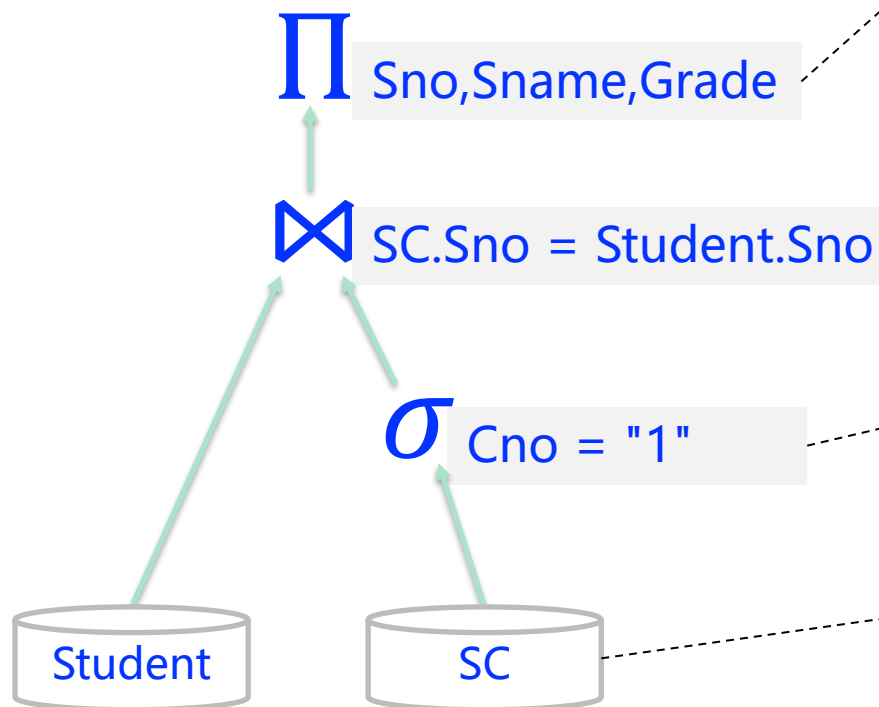
```
out = []
for t in Student: out.add(t)
return out
```

返回全部元组



物化模型执行流程

```
SELECT Sno, Sname, Grade
FROM SC, Student
WHERE SC.Sno = Student.Sno
AND Cno = "1"
```



```
out = []
for t in child.output():
    out.add(projection(t))
return out
```

```
out = []
for t1 in left.output():
    BuildHashTable(t1)
for t2 in right.output():
    if probe(t2): out.add(t1 ⋈ t2)
return out
```

```
out = []
for t in child.output():
    if evalPred(t): out.add(t)
return out
```

```
out = []
for t in SC: out.add(t)
return out
```

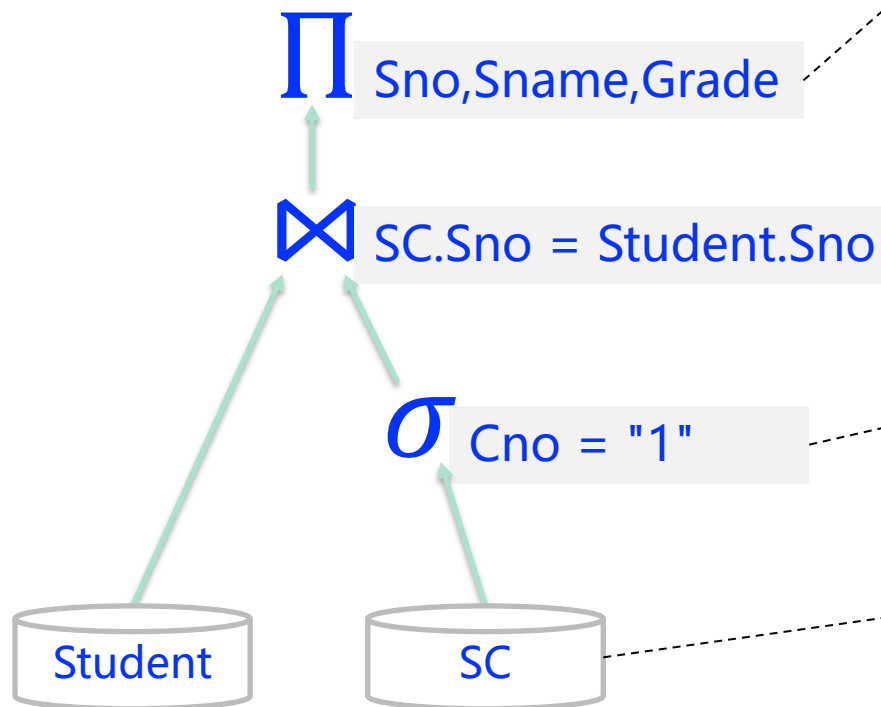
```
out = []
for t in Student: out.add(t)
return out
```

返回全部元组



物化模型执行流程

```
SELECT Sno, Sname, Grade
FROM SC, Student
WHERE SC.Sno = Student.Sno
AND Cno = "1"
```



```
out = []
for t in child.output():
    out.add(projection(t))
return out
```

```
out = []
for t1 in left.output():
    BuildHashTable(t1)
for t2 in right.output():
    if probe(t2): out.add(t1 ⋈ t2)
return out
```

```
out = []
for t in child.output():
    if evalPred(t): out.add(t)
return out
```

```
out = []
for t in SC: out.add(t)
return out
```

```
out = []
for t in Student: out.add(t)
return out
```

返回全部元组



向量化执行模型

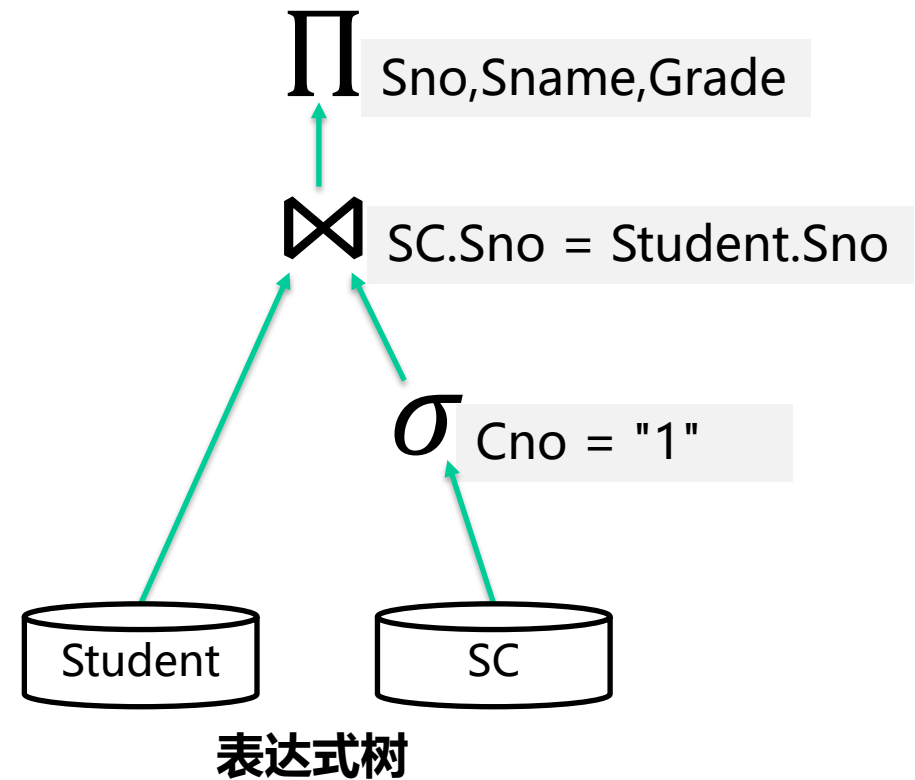
- 向量化执行模型 (Vectorization Model) 可以在一定程度上被视为火山模型与物化模型的中间形态;
- 通过next()函数自上而下地拉取数据, 每次会拉取一个批次的元组数据, 而不是只有一个。



向量化执行模型示例

- 查询获取“选修了1号课程的学生学号、姓名及成绩”

```
SELECT Sno, Sname, Grade
FROM SC, Student
WHERE SC.Sno = Student.Sno
AND Cno = "1"
```

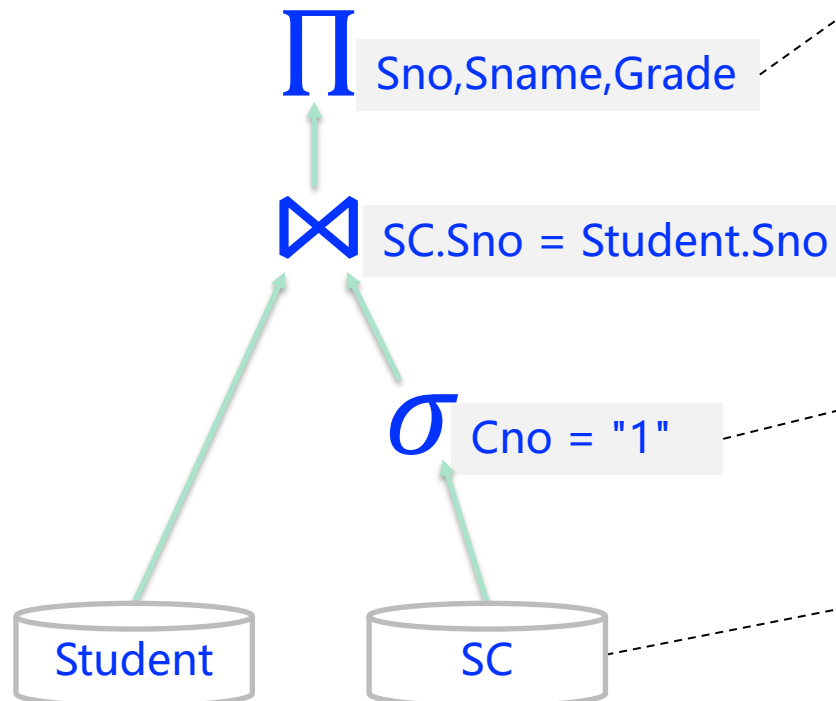




向量化执行模型执行流程



```
SELECT Sno, Sname, Grade
FROM SC, Student
WHERE SC.Sno = Student.Sno
AND Cno = "1"
```



```
out = []
for t in child.next():
    out.add(projection(t))
    if |out|>n: emit(out)
```

```
out = []
for t1 in left.next():
    BuildHashTable(t1)
for t2 in right.next():
    if probe(t2): out.add(t1 ⋈ t2)
    if |out|>n: emit(out)
```

```
out = []
for t in child.next():
    if evalPred(t): out.add(t)
    if |out|>n: emit(out)
```

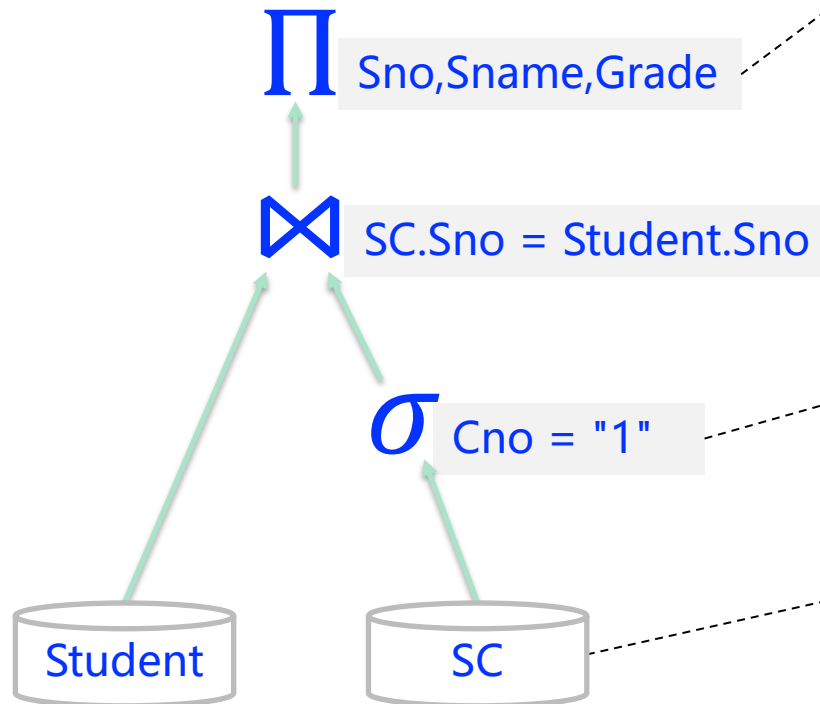
```
out = []
for t in SC:
    out.add(t)
    if |out|>n: emit(out)
```

```
out = []
for t in Student:
    out.add(t)
    if |out|>n: emit(out)
```



向量化执行模型执行流程

```
SELECT Sno, Sname, Grade  
FROM SC, Student  
WHERE SC.Sno = Student.Sno  
AND Cno = "1"
```



```
out = []  
for t in child.next():  
    out.add(projection(t))  
if |out| > n: emit(out)
```

```
out = []  
for t1 in left.next():  
    BuildHashTable(t1)  
for t2 in right.next():  
    if probe(t2): out.add(t1 ⋈ t2)  
if |out| > n: emit(out)
```

```
out = []  
for t in child.next():  
    if evalPred(t): out.add(t)  
if |out| > n: emit(out)
```

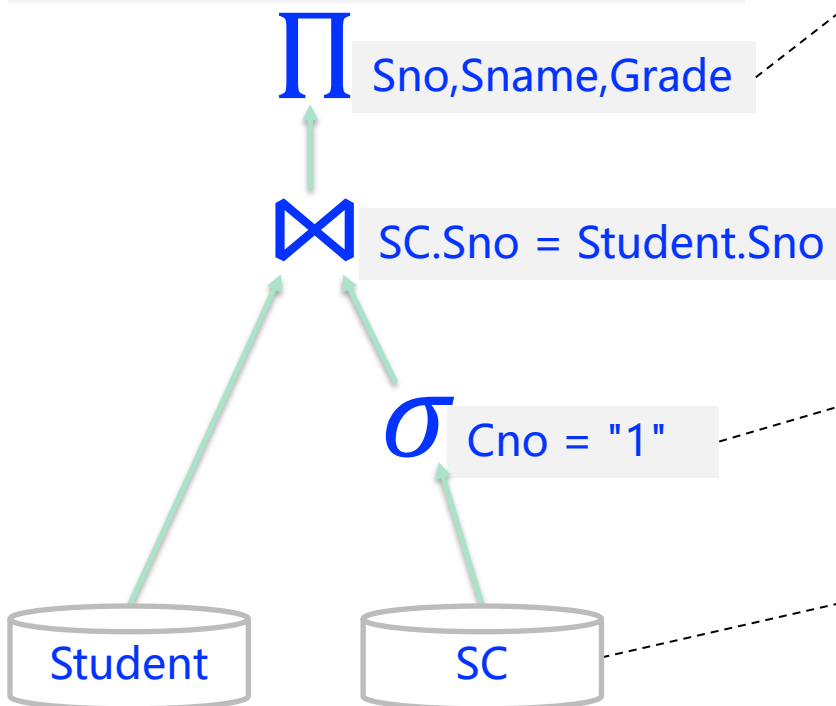
```
out = []  
for t in SC:  
    out.add(t)  
if |out| > n: emit(out)
```

```
out = []  
for t in Student:  
    out.add(t)  
if |out| > n: emit(out)
```



向量化执行模型执行流程

SELECT Sno, Sname, Grade
FROM SC, Student
WHERE SC.Sno = Student.Sno
AND Cno = "1"



```
out = []  
for t in child.next():  
    out.add(projection(t))  
if |out|>n: emit(out)
```

```
out = []  
for t1 in left.next():  
    BuildHashTable(t1)  
for t2 in right.next():  
    if probe(t2): out.add(t1 ⋈ t2)  
    if |out|>n: emit(out)
```

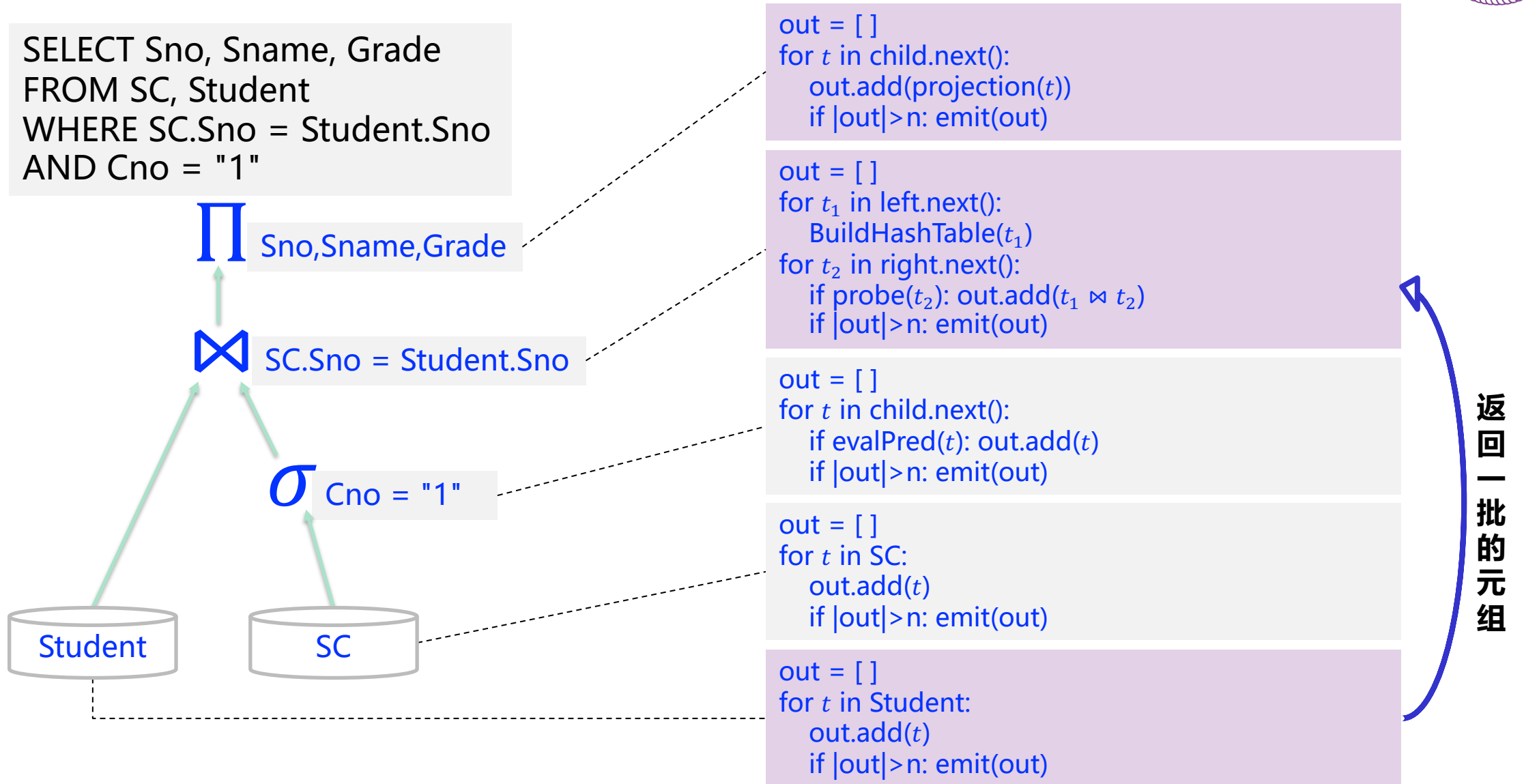
```
out = []  
for t in child.next():  
    if evalPred(t): out.add(t)  
    if |out|>n: emit(out)
```

```
out = []  
for t in SC:  
    out.add(t)  
    if |out|>n: emit(out)
```

```
out = []  
for t in Student:  
    out.add(t)  
    if |out|>n: emit(out)
```



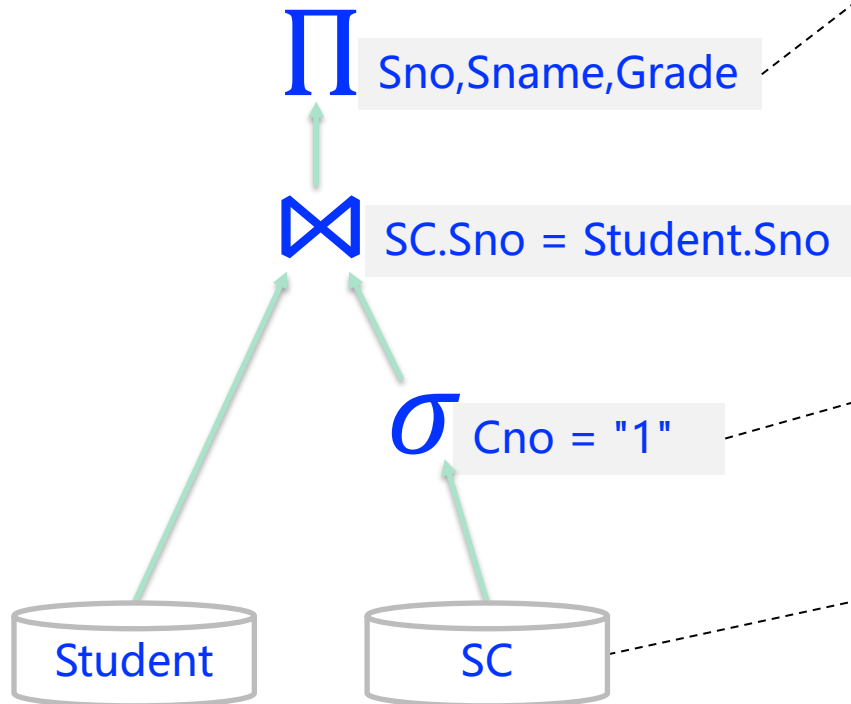
向量化执行模型执行流程





向量化执行模型执行流程

SELECT Sno, Sname, Grade
FROM SC, Student
WHERE SC.Sno = Student.Sno
AND Cno = "1"



```
out = []  
for t in child.next():  
    out.add(projection(t))  
if |out| > n: emit(out)
```

```
out = []  
for t1 in left.next():  
    BuildHashTable(t1)  
for t2 in right.next():  
    if probe(t2): out.add(t1 ⋈ t2)  
if |out| > n: emit(out)
```

```
out = []  
for t in child.next():  
    if evalPred(t): out.add(t)  
if |out| > n: emit(out)
```

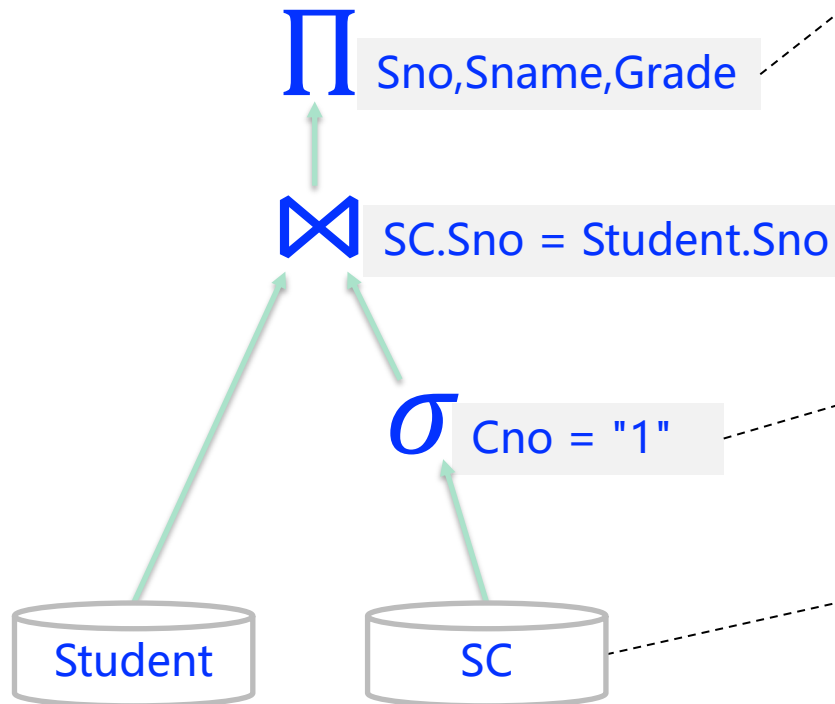
```
out = []  
for t in SC:  
    out.add(t)  
if |out| > n: emit(out)
```

```
out = []  
for t in Student:  
    out.add(t)  
if |out| > n: emit(out)
```



向量化执行模型执行流程

```
SELECT Sno, Sname, Grade
FROM SC, Student
WHERE SC.Sno = Student.Sno
AND Cno = "1"
```



```
out = []
for t in child.next():
    out.add(projection(t))
    if |out| > n: emit(out)
```

```
out = []
for t1 in left.next():
    BuildHashTable(t1)
for t2 in right.next():
    if probe(t2): out.add(t1 ⋈ t2)
    if |out| > n: emit(out)
```

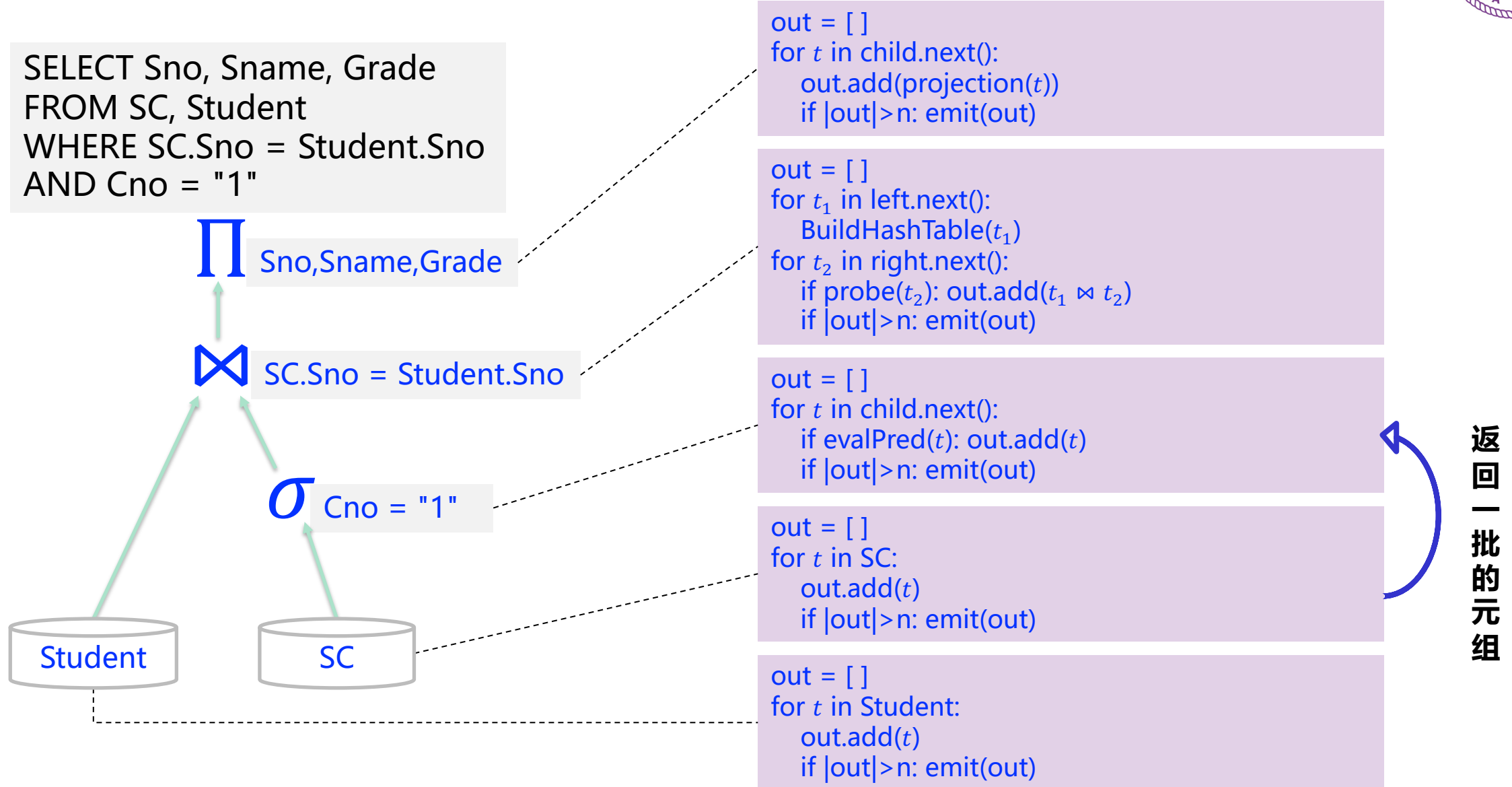
```
out = []
for t in child.next():
    if evalPred(t): out.add(t)
    if |out| > n: emit(out)
```

```
out = []
for t in SC:
    out.add(t)
    if |out| > n: emit(out)
```

```
out = []
for t in Student:
    out.add(t)
    if |out| > n: emit(out)
```



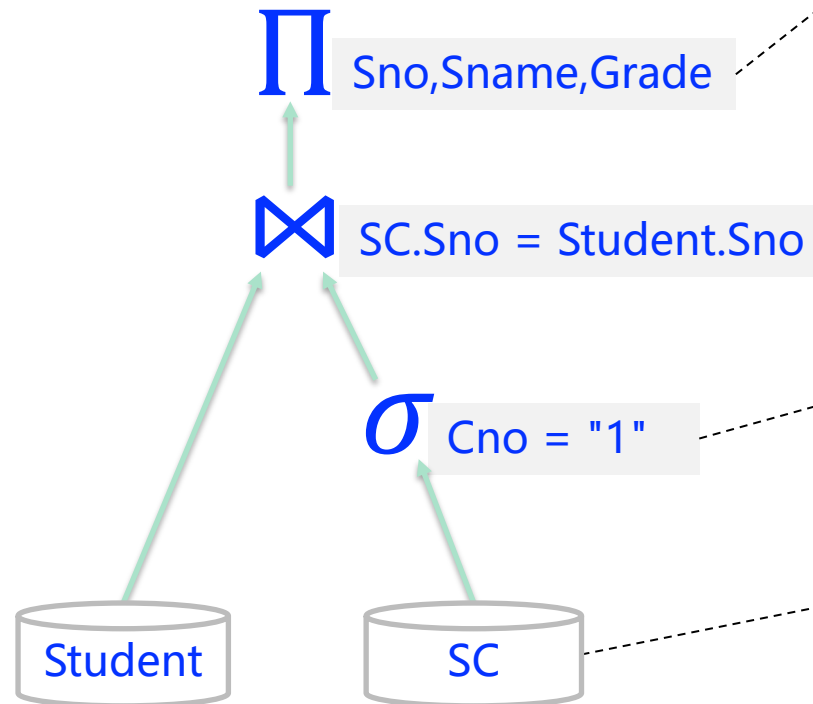

向量化执行模型执行流程





向量化执行模型执行流程

```
SELECT Sno, Sname, Grade  
FROM SC, Student  
WHERE SC.Sno = Student.Sno  
AND Cno = "1"
```



```
out = [ ]  
for t in child.next():  
    out.add(projection(t))  
if |out|>n: emit(out)
```

```
out = [ ]  
for t1 in left.next():  
    BuildHashTable(t1)  
for t2 in right.next():  
    if probe(t2): out.add(t1 ⋈ t2)  
    if |out|>n: emit(out)
```

```
out = [ ]  
for t in child.next():  
    if evalPred(t): out.add(t)  
    if |out|>n: emit(out)
```

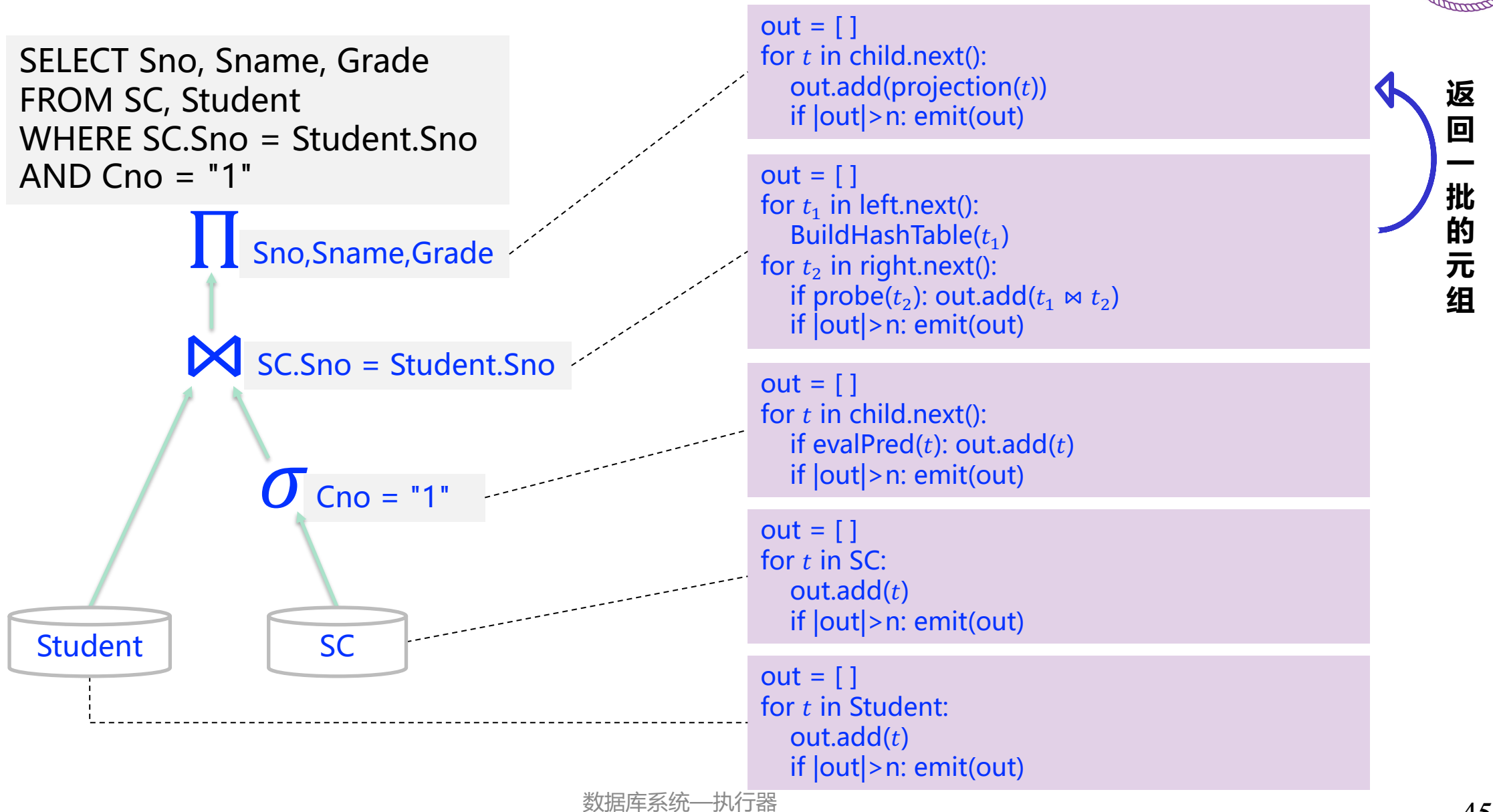
```
out = [ ]  
for t in SC:  
    out.add(t)  
    if |out|>n: emit(out)
```

```
out = [ ]  
for t in Student:  
    out.add(t)  
    if |out|>n: emit(out)
```

返回一批的元组



向量化执行模型执行流程





火山 vs 向量化 vs 物化

	火山	物化	向量化
函数调用	多	少	中
流水线	好	无	中
内存	小	大	中
适用场景	OLTP	OLTP	OLAP
代码局部性	差	好	中
数据局部性	中	好	好
实现难度	简单	中	难



目录

01

执行器概述

02

拉取式模型

03

推送式模型

04

并行执行

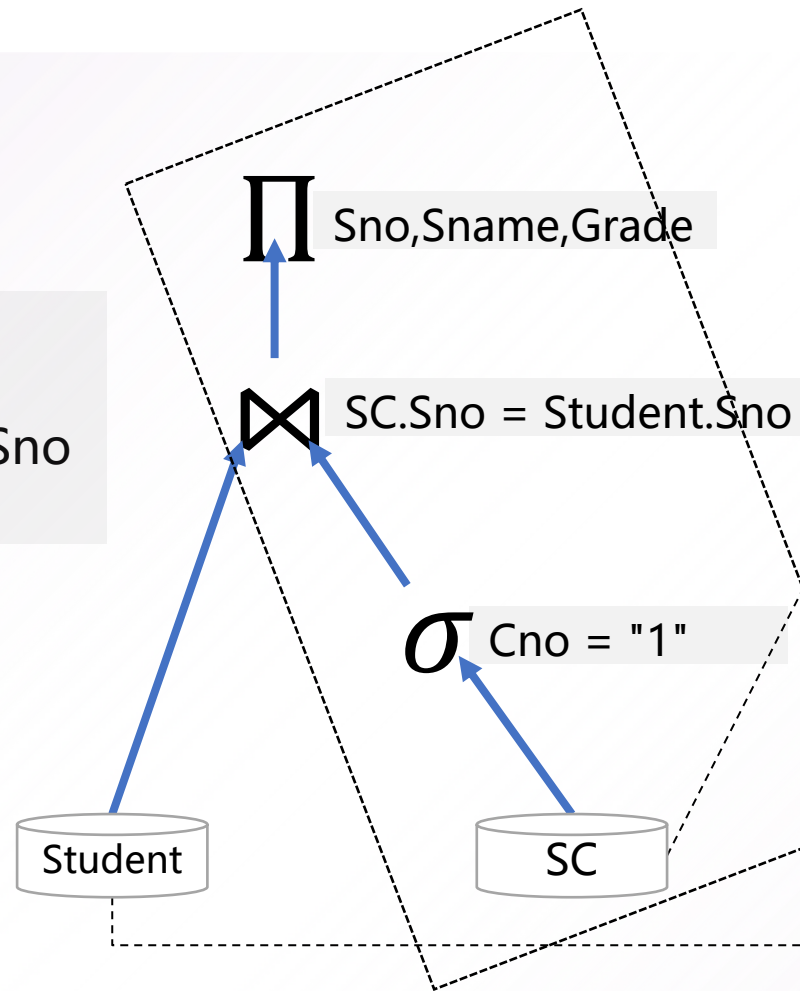
05

编译执行



推送式模型 Push-based

```
SELECT Sno, Sname, Grade  
FROM SC, Student  
WHERE SC.Sno = Student.Sno  
AND Cno = "1"
```



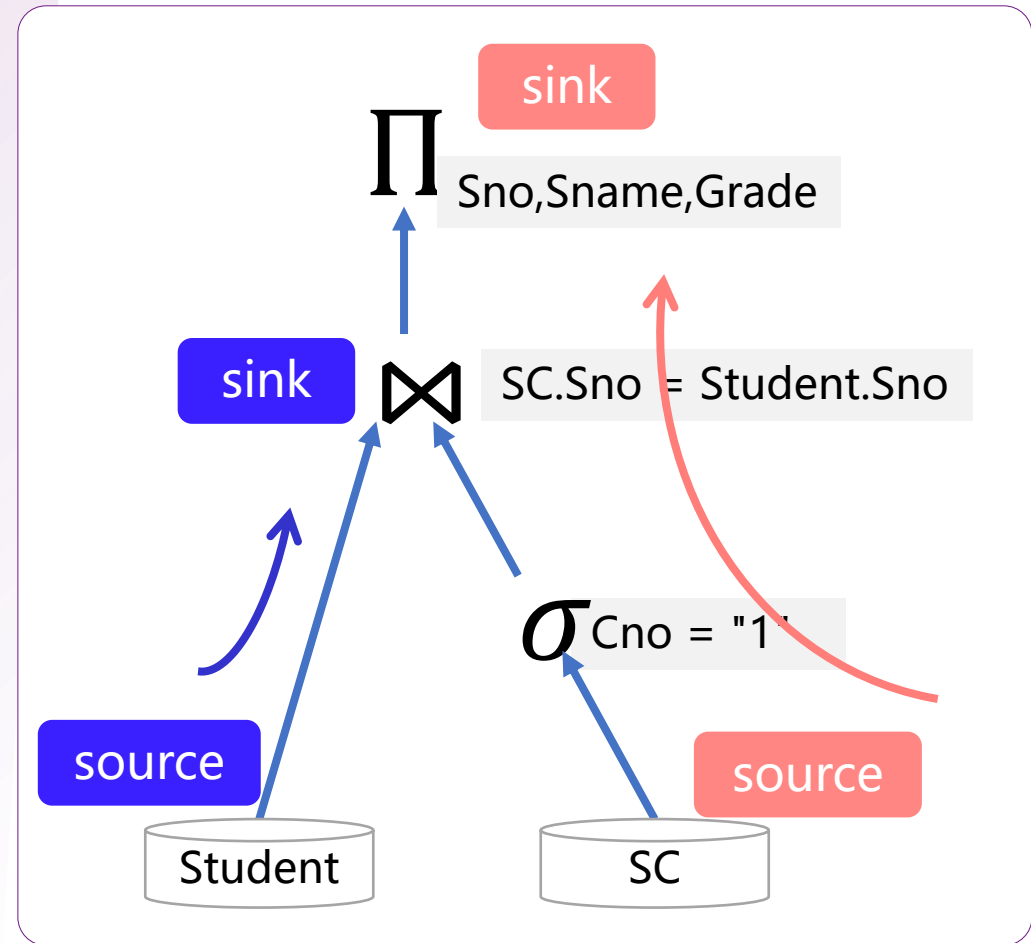
```
For  $t_2$  in SC:  
  if evalPred( $t_2$ ):  
    if probe( $t_2$ ) in HT:  
      //  $t_1$  in HT[ $t_2$ ]  
      emit( $t_1 \bowtie t_2$ )
```

```
for  $t_1$  in Student:  
  HT = BuildHashTable( $t_1$ )
```



推送式模型 Push-based

- 自底向上
- 遇到汇聚节点需要分解为不同的pipeline
 - 汇聚节点包括：
 - 连接Join
 - 哈希Hash
 - 排序Sorting
 - 分组Grouping
- Pipeline之间需要调度：并行或者先后依赖
 - 祖先后代pipeline有先后依赖
 - 无祖先后代pipeline可以并行
- 每个Pipeline自底向上执行





推送式模型 Push-based



➤ 根据物理计划分解Pipeline

- 每个pipeline分为三个部分
 - Source源节点：一般为扫描一个表或者一个中间结果(哈希、分组、排序)
 - Operator算子节点：查询计划树上的节点（过滤、投影、probe)
 - Sink汇聚节点：计算中间结果的物化节点（父pipeline的source)

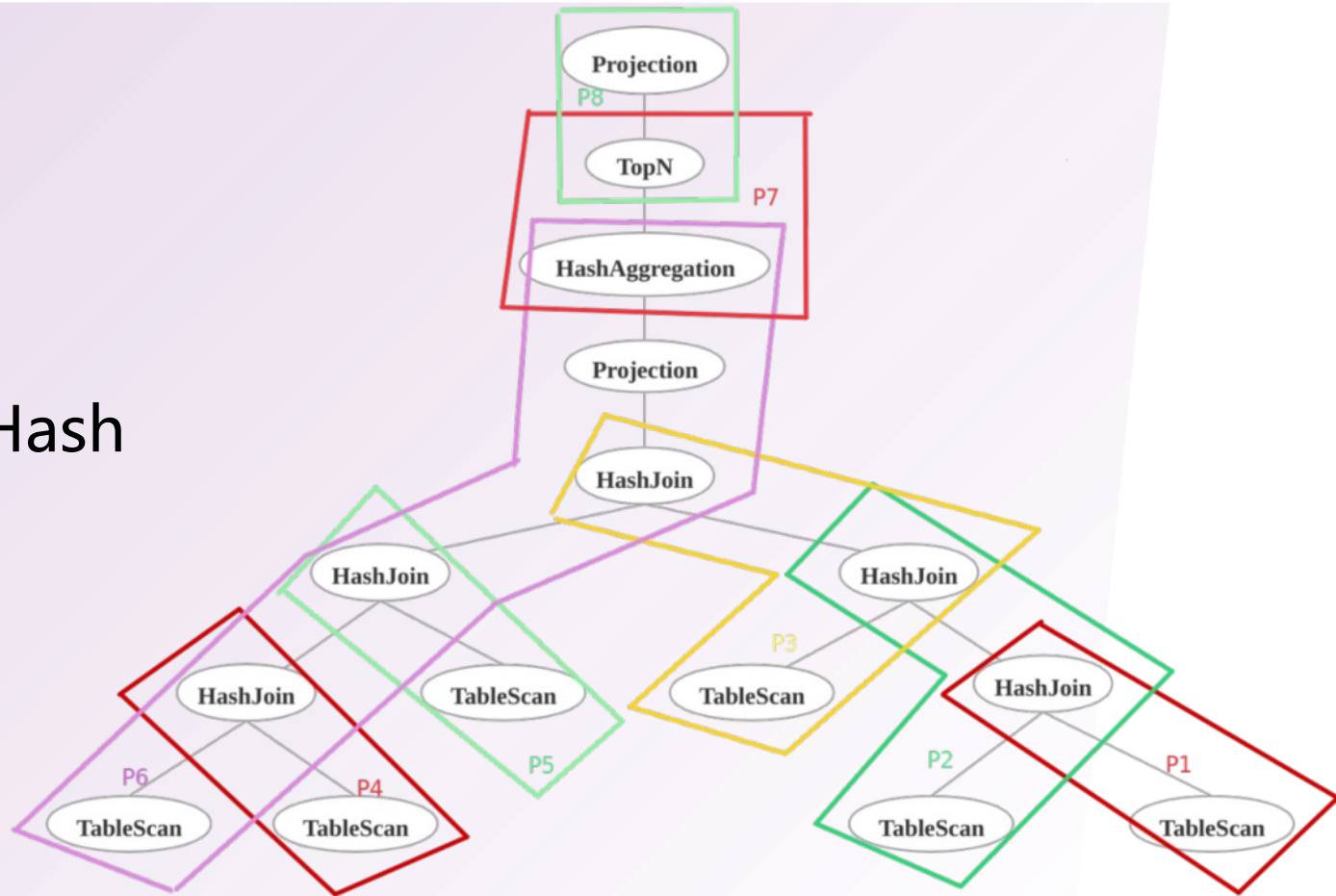
➤ 调度器调度pipeline

- 根据pipeline的关系调度pipeline
- 自底向上执行每个pipeline



Pipeline分解

- 枚举算子
- 找到sink节点
 - Join、Group、Sorting、Hash
- 根据sink节点生成pipeline
- 确定pipeline的先后关系





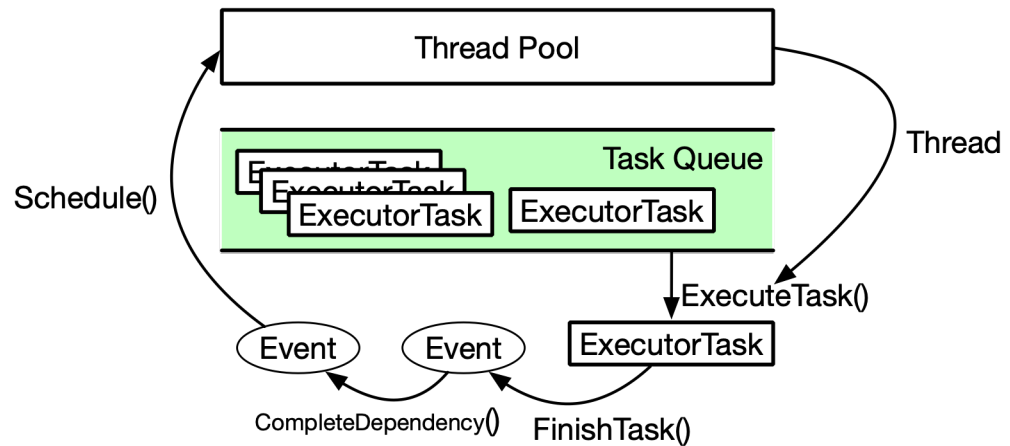
Pipeline调度

➤ 事件驱动的调度器

- 生产者消费者模型
- 任务队列
- 父子pipeline依赖调度

➤ 每类节点的执行状态

- Have_more_output, 例如扫描
- Need_more_input, 例如聚合
- Finished, 例如结束
- Blocked, 例如内存满



```
PipelineExecuteResult PipelineExecutor::Execute() {
    DataChunk source_chunk, final_chunk;
    std::vector<DataChunk> intermediate_chunks;
    while (true) {
        result = pipeline.source.GetData(source_chunk, source_state);
        for (int i = 0; i < pipeline.operators.size(); i++) {
            result = pipeline.operators[i].Execute(
                intermediate_chunks[i], intermediate_chunks[i+1], intermediate_states[i]);
        }
        result = pipeline.sink.Sink(final_chunk, sink_state);
        if (result == SinkResultType::FINISHED) {
            return PipelineExecuteResult::FINISHED;
        }
    }
}

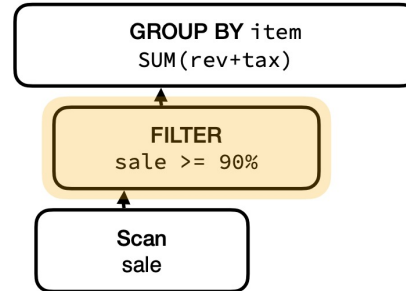
enum class SourceResultType { HAVE_MORE_OUTPUT, FINISHED, BLOCKED };
enum class OperatorResultType { NEED_MORE_INPUT, HAVE_MORE_OUTPUT, FINISHED, BLOCKED };
enum class SinkResultType { NEED_MORE_INPUT, FINISHED, BLOCKED };
```



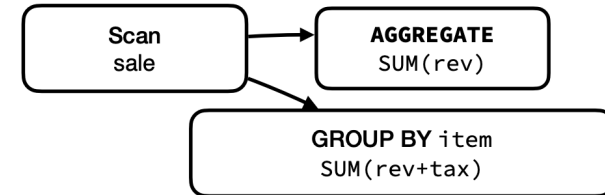
推送式模型 Push-based优点

- **Caching: 数据库复用**
- **扫描共享: 多个并发扫描一张表**
- **流控机制: Limit 操作**
- **异步IO: 网络或者大表**

Vector Cache
Add **small caches** between operators

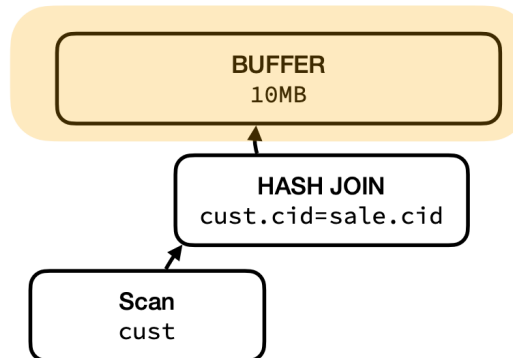


Scan Sharing
We can push results of one scan into multiple sinks



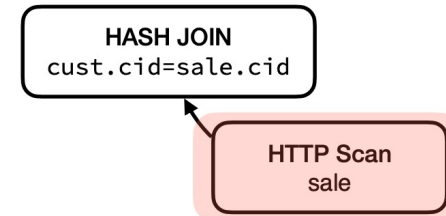
Backpressure

Pause pipeline when buffer is full
Resume when buffer is empty



Async I/O

Pause pipeline while HTTP request is in progress
Resume when data is available





推送式 vs 拉取式

	拉取式	推送式
开始节点	根节点	叶子节点
执行方向	自上而下	自下而上
核心	以算子为核心	以数据为核心
执行方式	通过函数从子节点“拉取数据”	将数据“推送”给父节点
优势	易实现、优雅、省内存	函数调用少、缓存命中率高、效率高
劣势	函数调用多、缓存命中率低	Limit、sort-merge join效率低，内存消耗高



目录

01

执行器概述

02

拉取式模型

03

推送式模型

04

并行执行

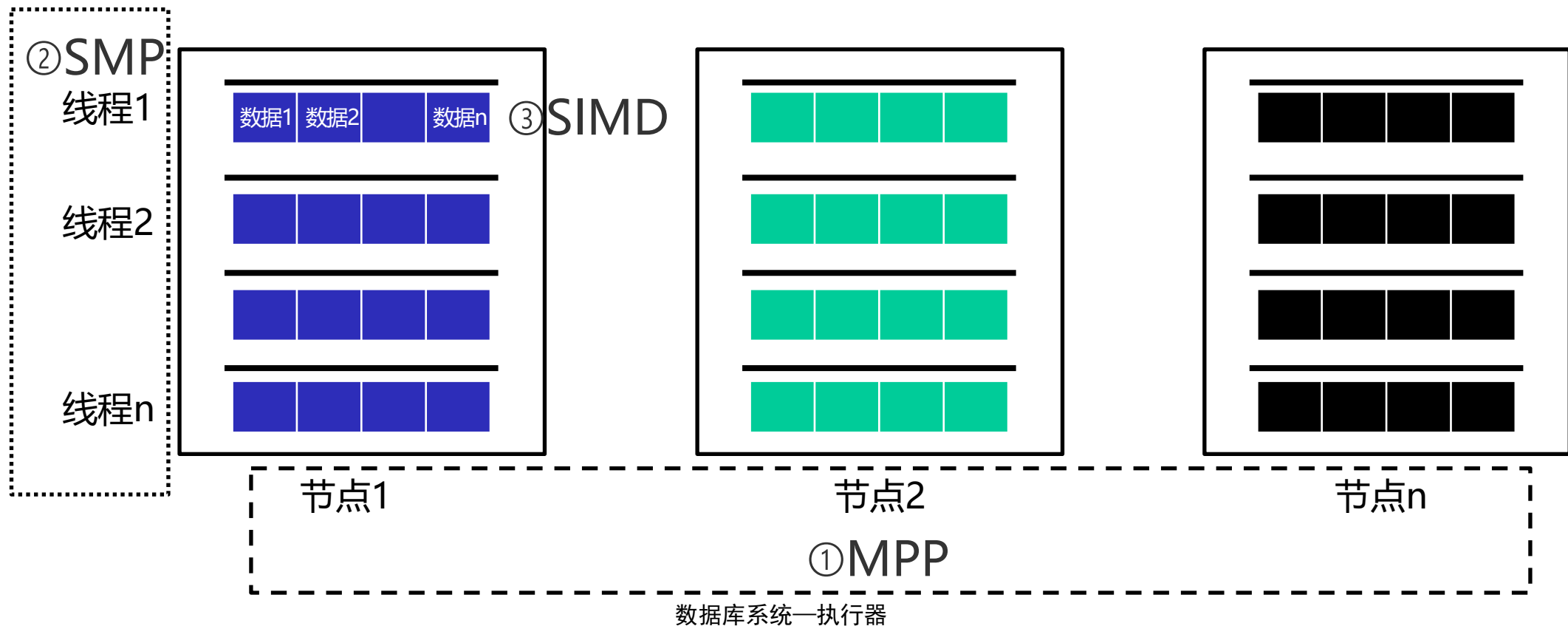
05

编译执行



并行执行

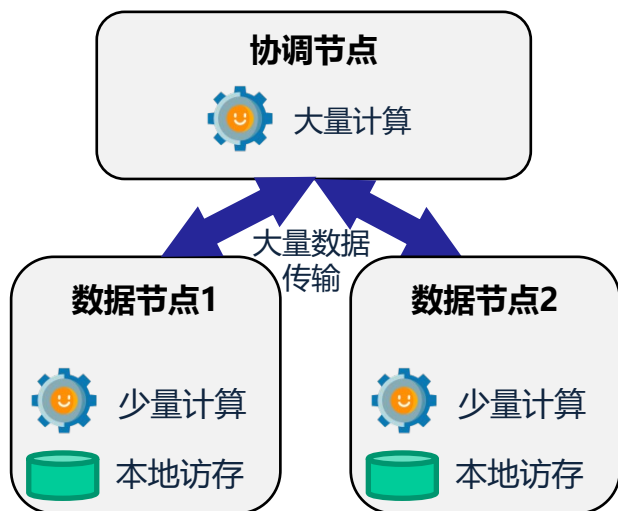
- ① 节点间多机器并行 MPP: Massively Parallel Processing
- ② 节点内多线程并行 SMP: Symmetric Multi-processing
- ③ 单指令多数据并行 SIMD: Single Instruction Multiple Data



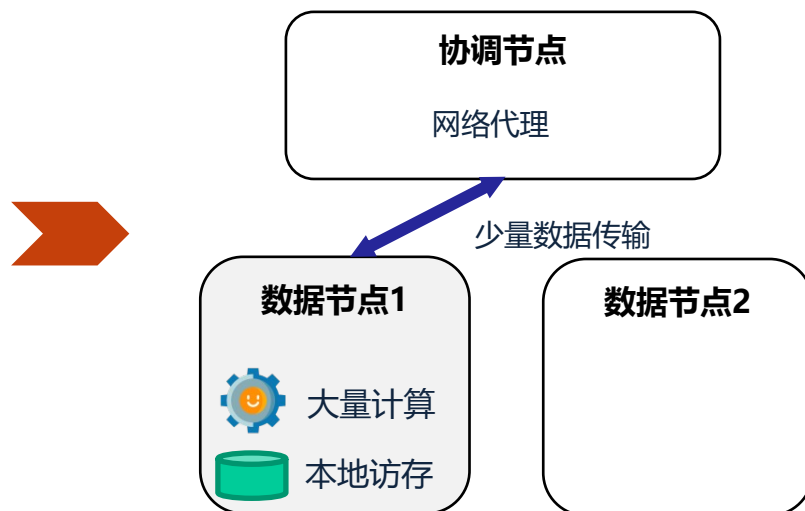


节点间多机器并行 MPP

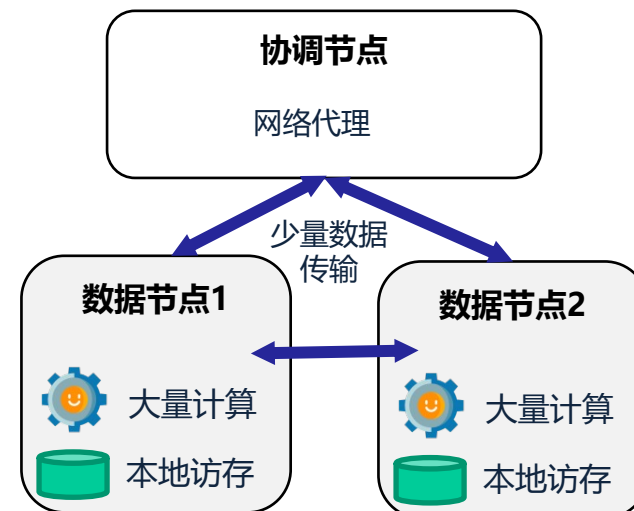
传统分发-汇聚的分布式计划



协调节点旁路技术



数据节点自协同的流式计划





节点内多线程并行 SMP

① 算子间并行

□ 计划切分

□ 流水线

② 算子内并行

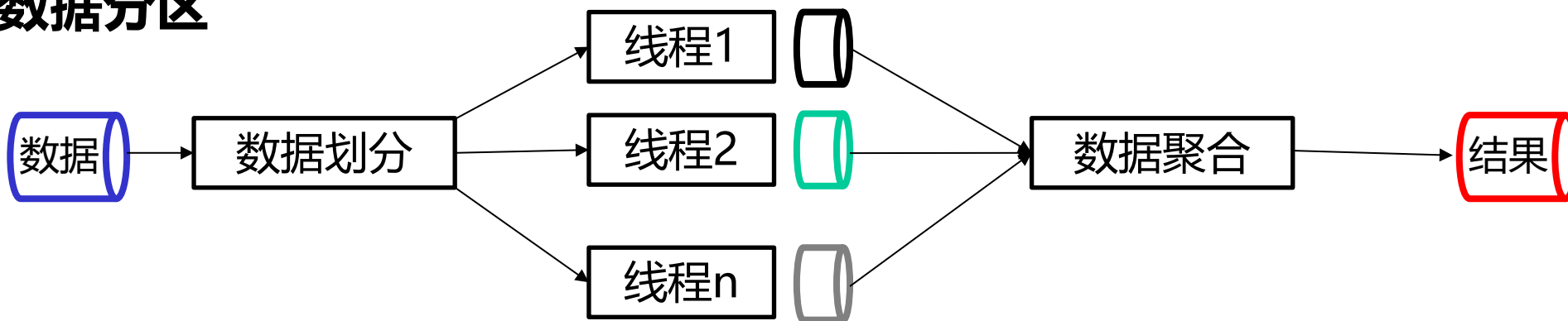
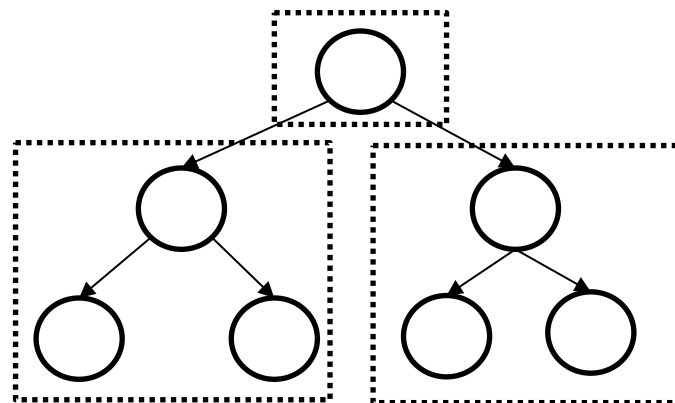
□ 算子

✓ 扫描、插入

✓ Group、aggregation、sort

✓ Join

□ 数据分区





节点内多线程并行 SMP

① 算子间并行

□ 计划切分

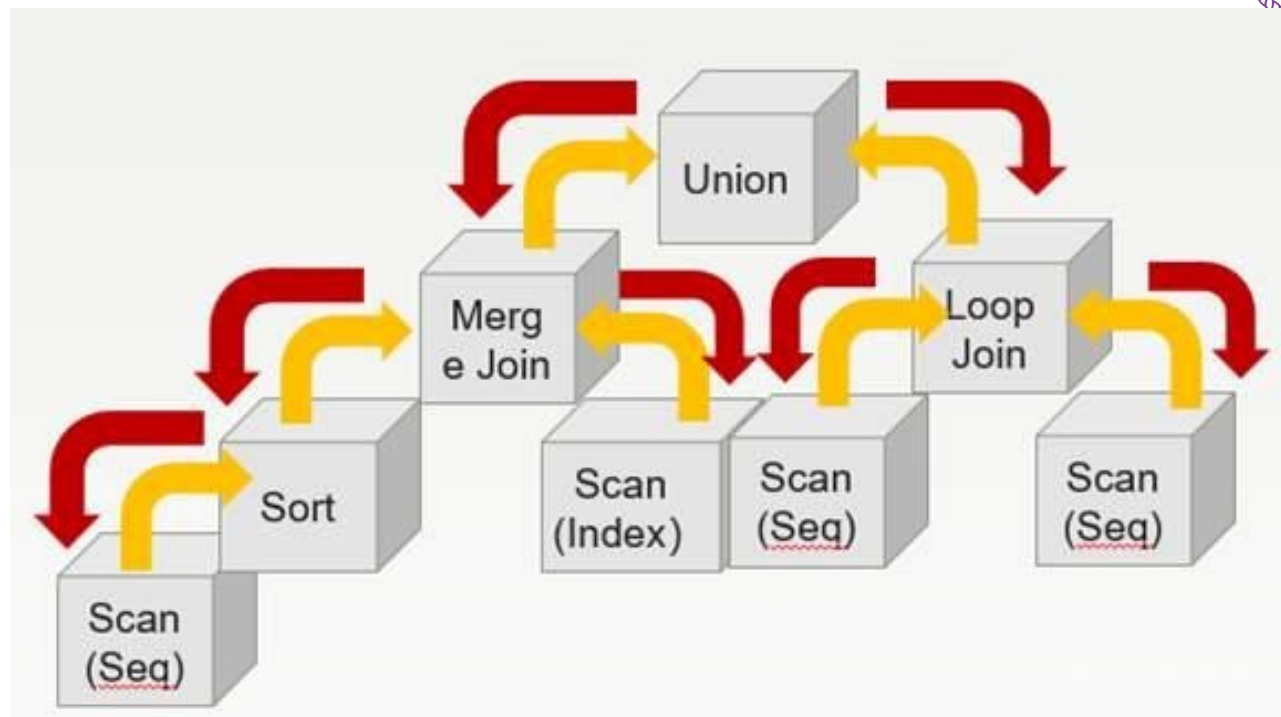
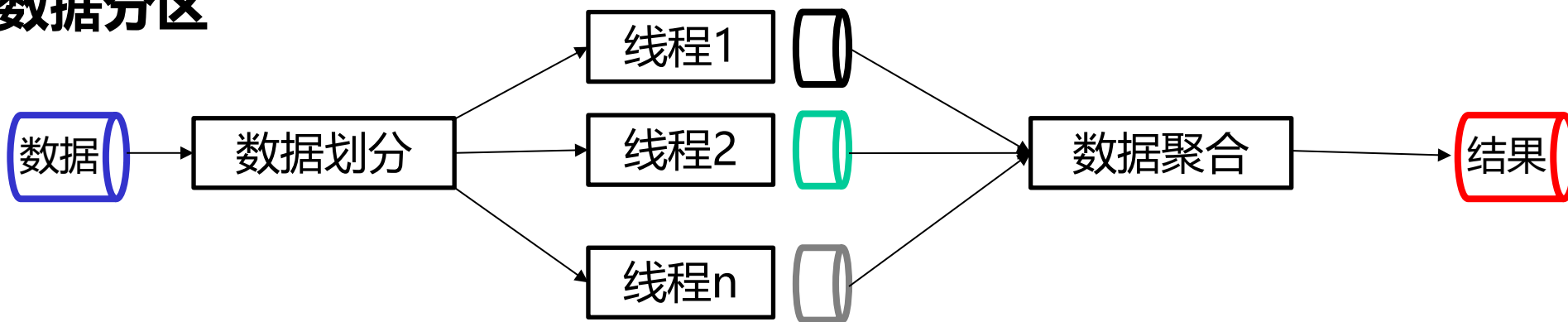
□ 流水线

② 算子内并行

□ 算子

- ✓ 扫描、插入
- ✓ Group、aggregation、sort
- ✓ Join

□ 数据分区

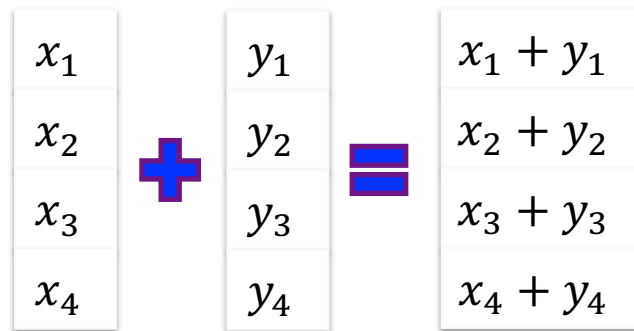




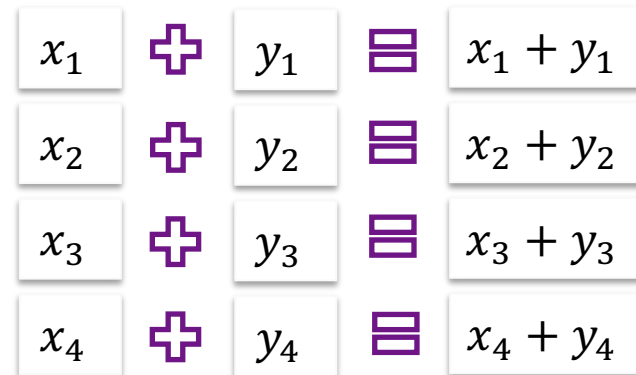
单指令流多数据 SIMD

- 单指令流多数据流 (single instruction multiple data, SIMD) 是一种允许同一个处理器在多个数据点上执行相同操作, 进而形成并行计算的技术。

SIMD模式



标量模式





SIMD支持的部分指令

指令	示例
数据移动	将数据移入或移除寄存器
算术运算	ADD, SUB, MUL, DIV, SQRT, MAX, MIN
逻辑指令	AND, OR, XOR, ANDN, ANDPS, ANDNPS
比较指令	==, <, <=, >, >=, !=
Shuffle指令	在寄存器之间转移数据
其他	转换指令：在x86和SIMD寄存器之间转换数据； 缓存控制：将数据从SIMD寄存器中直接转移到内存中



并行执行：Morsel-driven

□ NUMA和SMP情况下，如何并行处理数据？

□ 如何解决负载均衡问题？

□ Morsel-driven执行框架

- Morsel (小块)：最小的调度单位，通常包含数千行或一个数据块 (chunk)。它将原本庞大的数据，切割成大量独立、可并行处理的单元，为后续的负载均衡提供了基础。
- Pipeline Breaker (流水线断点)：查询计划中的“分界线”。像排序 (Sort)、哈希聚合 (Hash Aggregation) 算子，必须拿到所有数据才能工作，会打断执行流水线。而选择 (Select)、投影 (Project) 等算子则不会。流水线断点将查询计划自然地划分为多个连续阶段。
- Pipeline (流水线)：这是执行工作的基本单位。一个 Pipeline 由查询计划中两个连续的 Pipeline Breaker 之间的所有非阻塞算子组成。它们可以作为一个整体被调度执行。



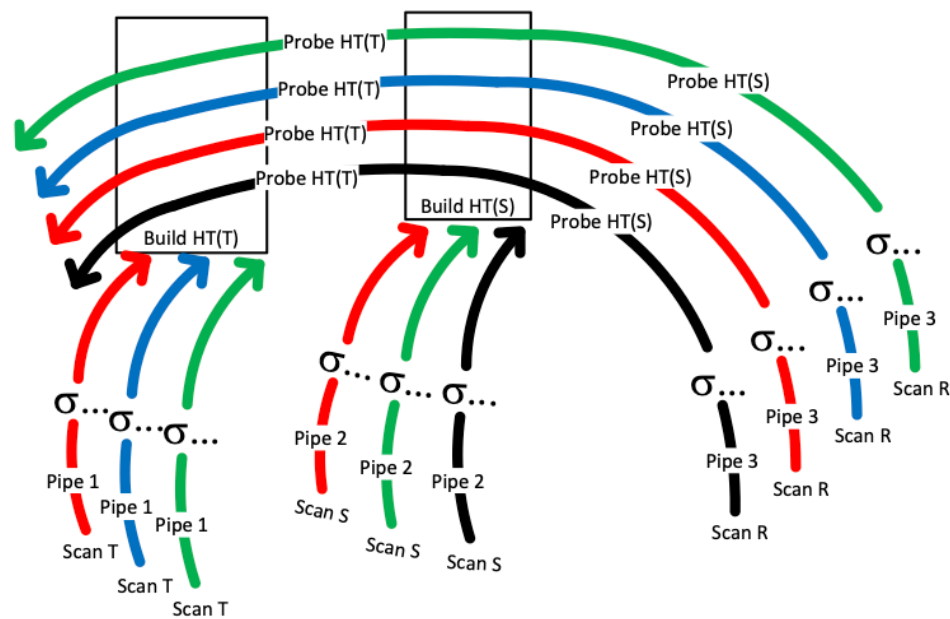
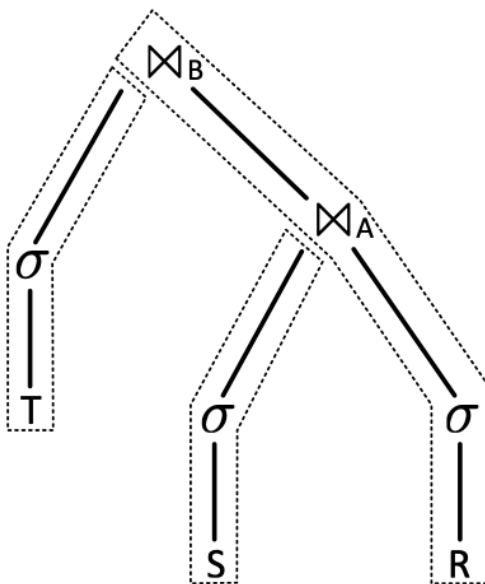
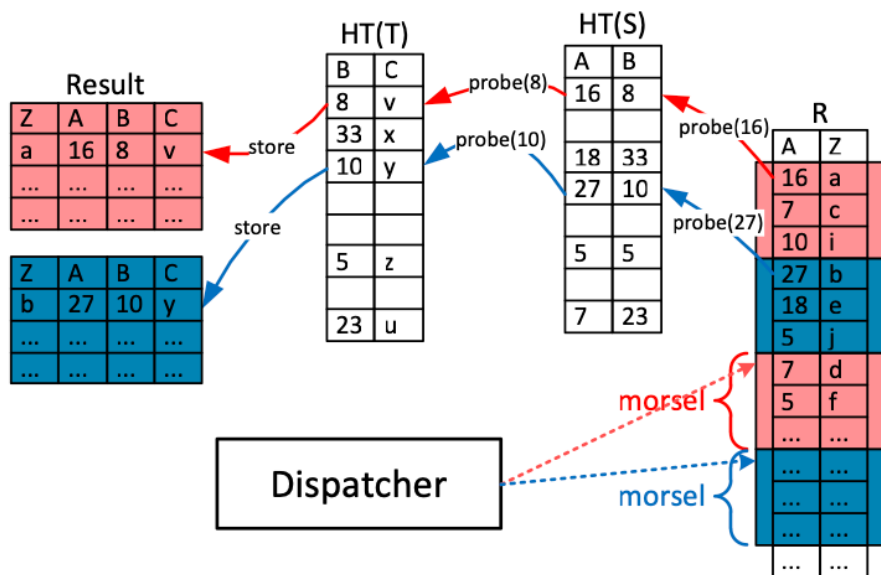
并行执行Morsel-driven: 架构

- ① 查询计划分解 (Plan Decomposition) : 以Pipeline Breaker为界, 被切分成多个独立的执行阶段 (Pipeline)
- ② Morsel生成 (Morsel Generation) : 在执行阶段 (Pipeline), 数据源 (如 TableScan 算子) 会将被处理的数据表按行切分成大量细粒度的Morsel。
- ③ Pipeline执行与动态调度 (Pipeline Execution & Dynamic Scheduling)
 - 线程预分配与核心绑定: 系统会为每个CPU核心预分配一个工作线程 (Worker), 并通常会与 NUMA节点绑定, 以实现更好的NUMA亲和性。
 - Morsel分发: Morsel会被持续分发到空闲的线程上。一个线程获取一个Morsel后, 从头到尾执行完其所属的整个Pipeline, 直到遇到下一个Pipeline Breaker。
 - 工作窃取 (Work-Stealing) : 工作线程处理完自己的Morsel后, 会检查全局队列, 甚至主动从其他繁忙线程的队列末尾“窃取”一个Morsel来执行。
- ④ 阻塞算子与Pipeline Breaker处理
 - Sink (汇聚) : 各线程在本地对各自的Morsel数据进行排序或构建哈希表。
 - Combine (合并) : 线程完成本地处理后, 进入等待同步状态



并行执行Morsel-driven: 案例

- 数据驱动，数据切片，并行调度
- NUMA并行执行，保护 NUMA 本地性。每个核只负责处理其所在CPU核的数据。
- 任务调度、窃取，查询的并行度完全灵活。
- 负载均衡。 task 的动态分配机制允许 task 的抢占，执行较快的核可以抢占执行较慢的核的 task，避免了等待空转的现象。





并行执行Morsel-driven: 哈希、排序



□ 思想：分而治之，局部先行，全局合并

□ 基于Morsel的哈希连接 (Partitioned Hash Join)

- 分区阶段 (Partition Phase): 所有线程并行计算所有输入数据的哈希值，根据高位快速将数据切分到不同的逻辑分区中，为无冲突并行打好基础。
- 构建与探测 (Build & Probe): 为每个分区分配一个或多个线程，在每个分区内，线程独立地执行串行的哈希连接（构建哈希表 → 探测哈希表 → 输出结果），彻底避免了线程间的同步开销。
- 合并结果 (Combine): 将各个分区产出的结果合并，得到最终结果集。

□ 基于Morsel的排序

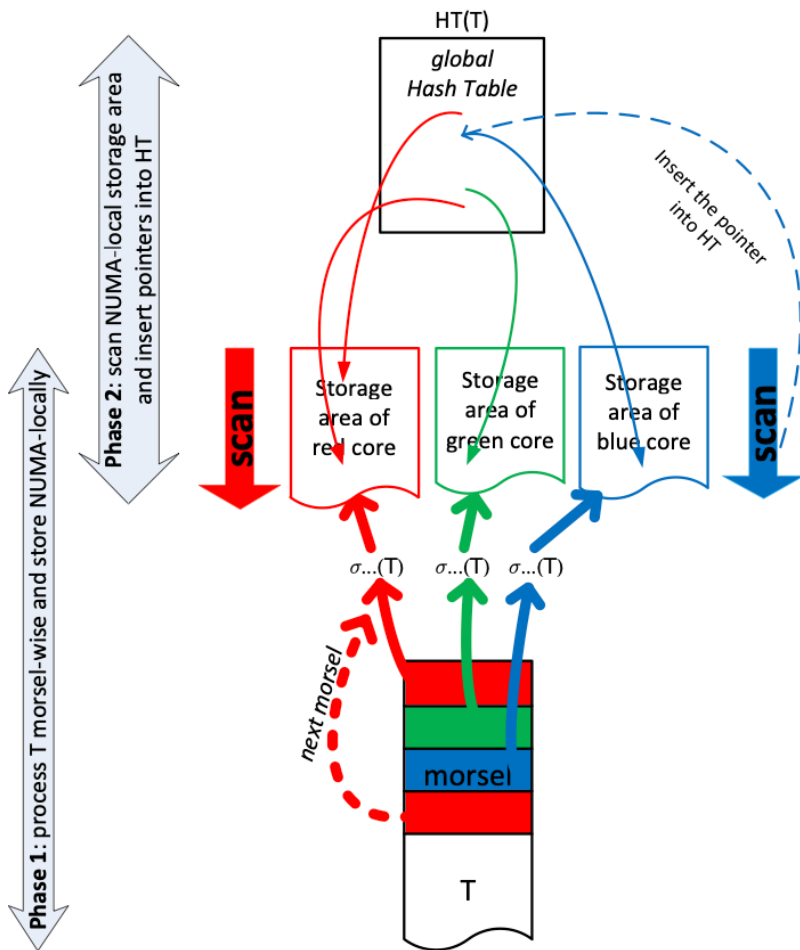
- 并行局部排序 (Parallel Local Sort): 在“Sink”阶段，多个线程并行处理各自的Morsel数据块，各线程生成多个内部有序的“归并段 (Sorted Run)”。
- 全局并行归并 (Parallel Global Merge): 当所有线程完成局部排序后，系统进入“Finalize”或“GetData”阶段，执行全局归并。



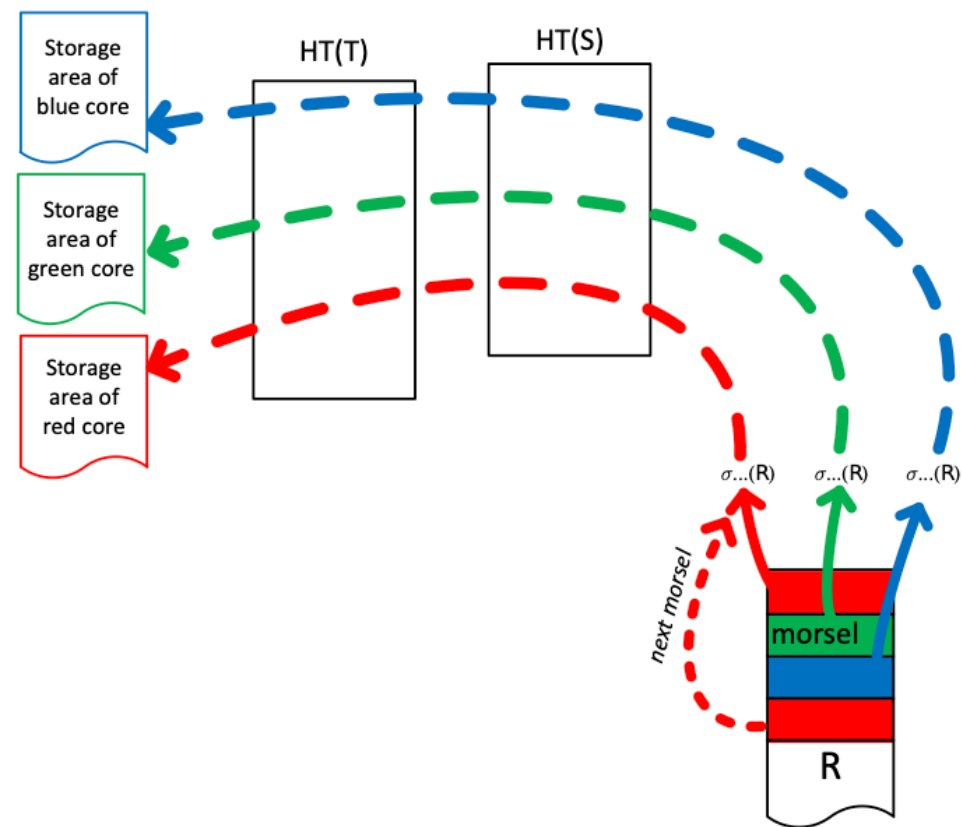
并行执行Morsel-driven: Hash Join



□ Hash Join的Build阶段



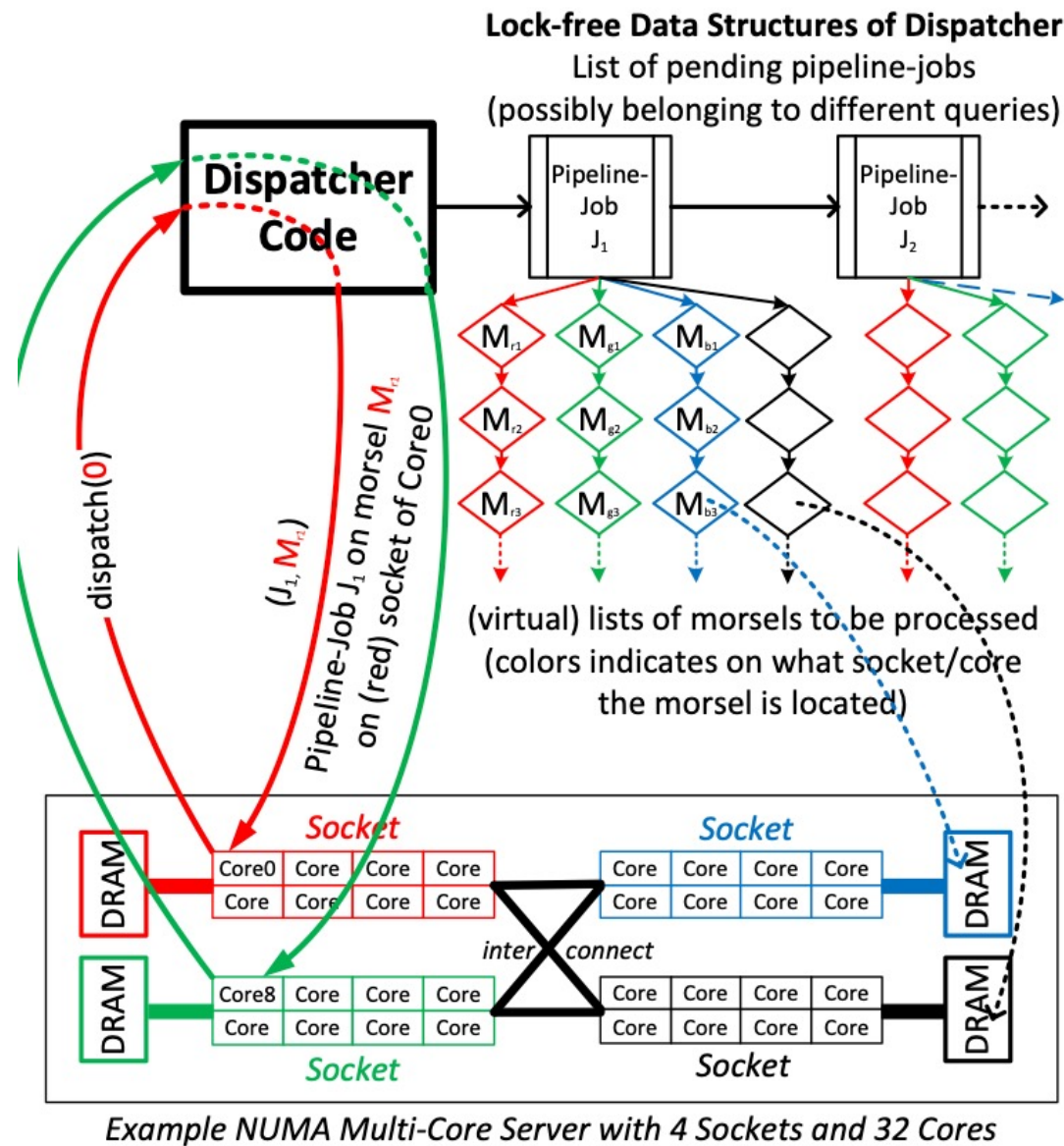
□ Hash Join的Probe阶段





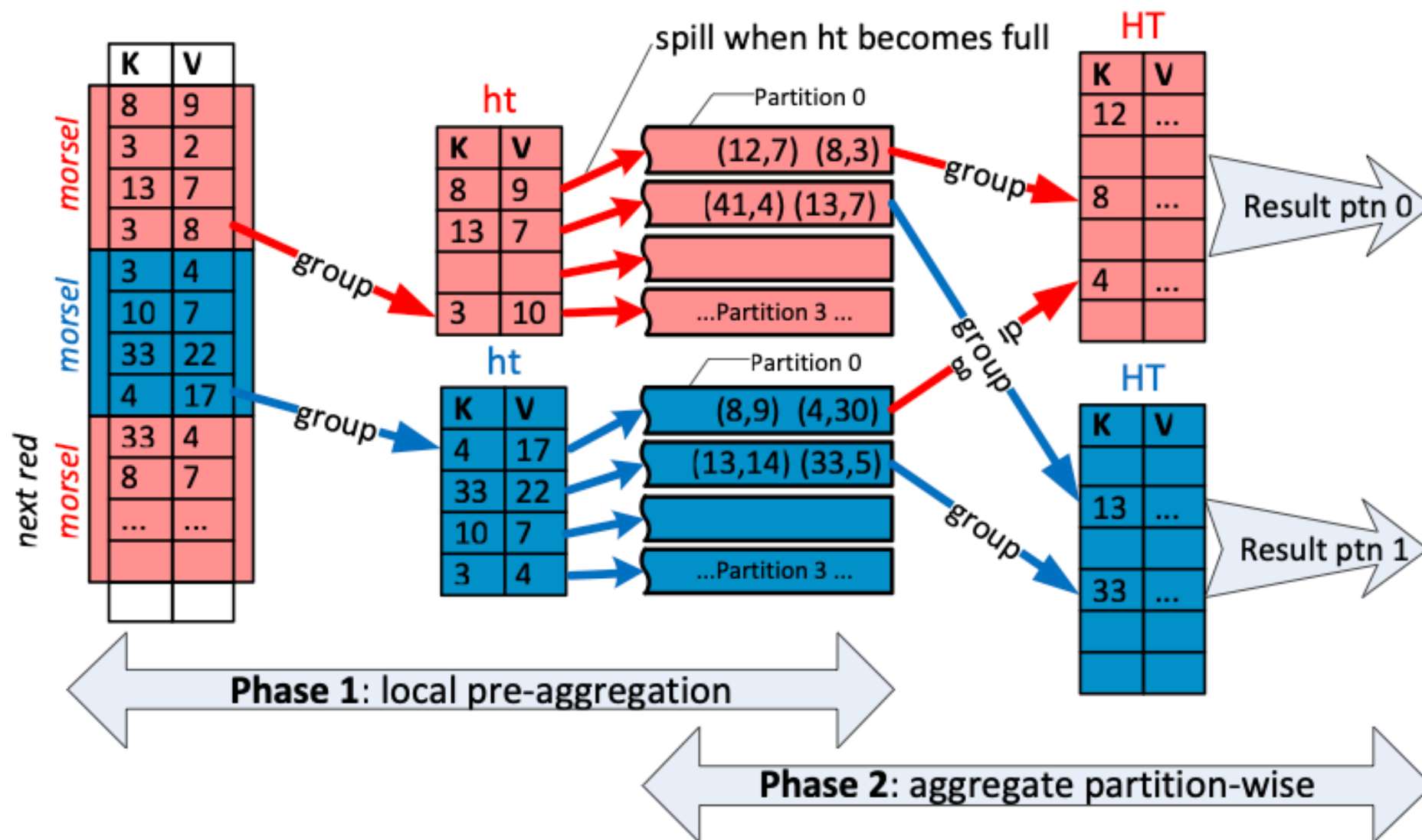
并行执行Morsel-driven: 例子

- ① 生成pipeline
- ② 生成morsel
- ③ Dispatcher 负责分配任务给 worker 线程
- ④ 动态分配 task 的机制保证了查询并行度的灵活性。



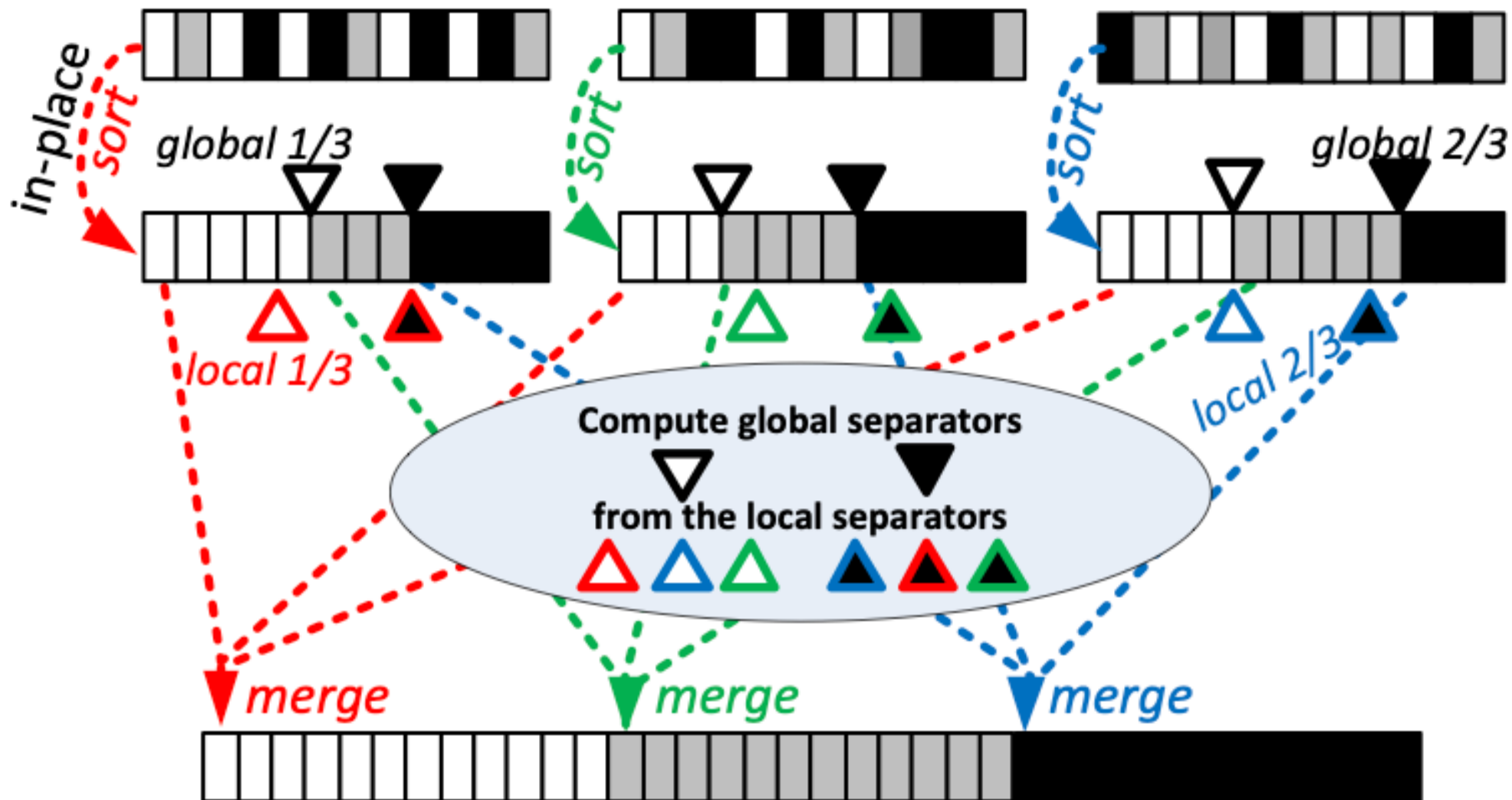


并行执行Morsel-driven: 分组聚集





并行执行Morsel-driven: Sort-merge Join





目录

01

执行器概述

02

拉取式模型

03

推送式模型

04

并行执行

05

编译执行



编译执行

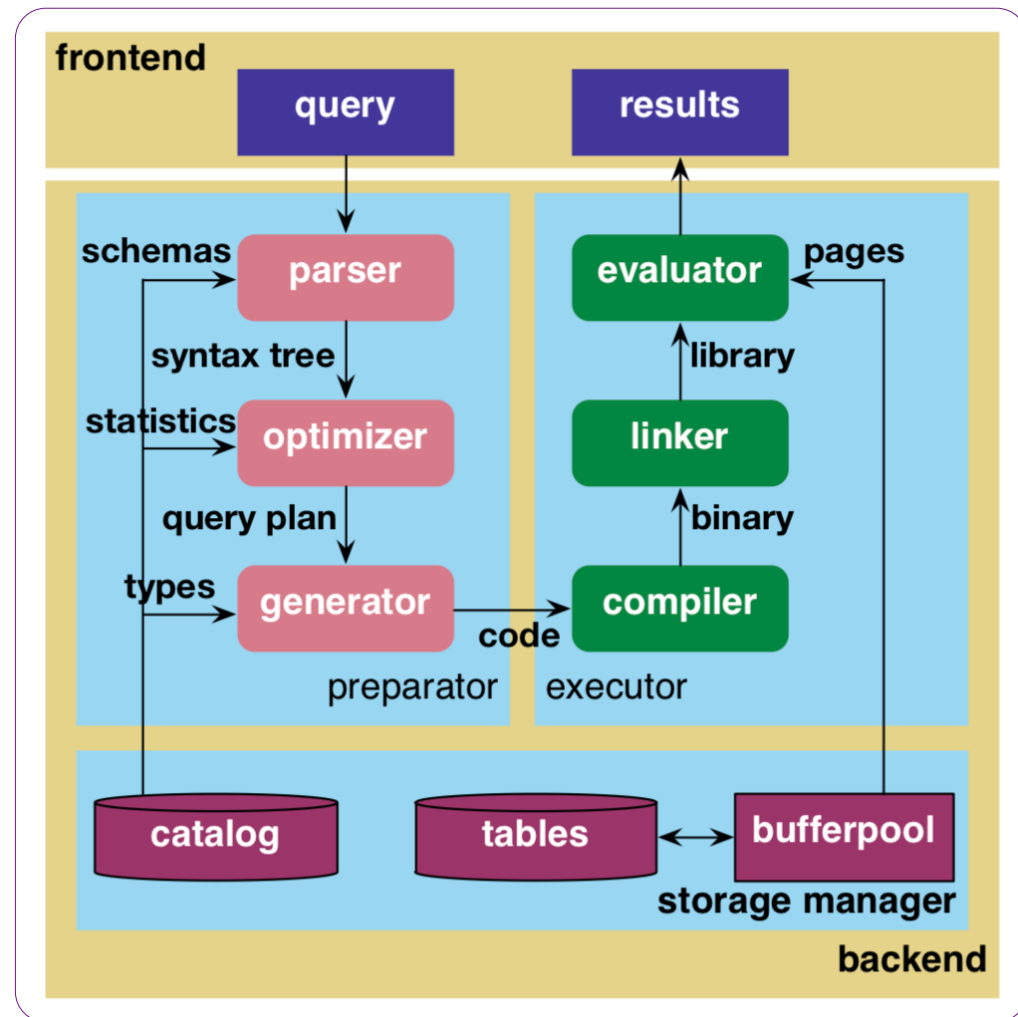
- 通过**编译执行技术避免大量虚函数调用**，并裁剪了大量执行中的分支判断等指令，减少了无用指令数量；
- 在火山模型中，每次获取数据都需要访问一次内存，编译执行避免迭代调用过程，通过多级存储和预取技术将数据缓存在CPU，提升了缓存命中率；
- 循环展开和优化，通过流水线和指令重排序技术，提升了循环的运行效率。



预先编译与实时编译



- **预先编译：**数据库中的“预先编译”通常指预编译语句或存储过程的提前编译（相对于执行时刻），但编译动作仍然发生在数据库运行时（首次调用时），编译结果被缓存复用。其优点是减少后续执行的解析和优化开销。
- **实时编译：**数据库中的实时编译（JIT）是指在查询执行过程中，将**表达式计算**、**元组解构**等解释执行的热点代码动态编译为机器码，以消除解释开销。其触发时机由成本估算和执行频率决定，目的是在编译开销和执行性能之间取得平衡。





编译执行

➤ 两种编译执行方法

① HIQUE: 查询执行计划 → 高级语言代码 (例如C) → 可执行代码

- 将SQL查询转化为C语言源码, 再调用外部编译器编译成机器码
- 离线AOT (提前编译): 生成完整C后调用编译器, 编译完成后才能在数据库中执行。
- 编译代价高: 涉及生成C代码、写入文件、启动外部编译器进程、链接等操作, 适合一次性编译、多次执行的场景

② Hyber: 查询执行计划 → 中间表示IR (LLVM) → 可执行代码

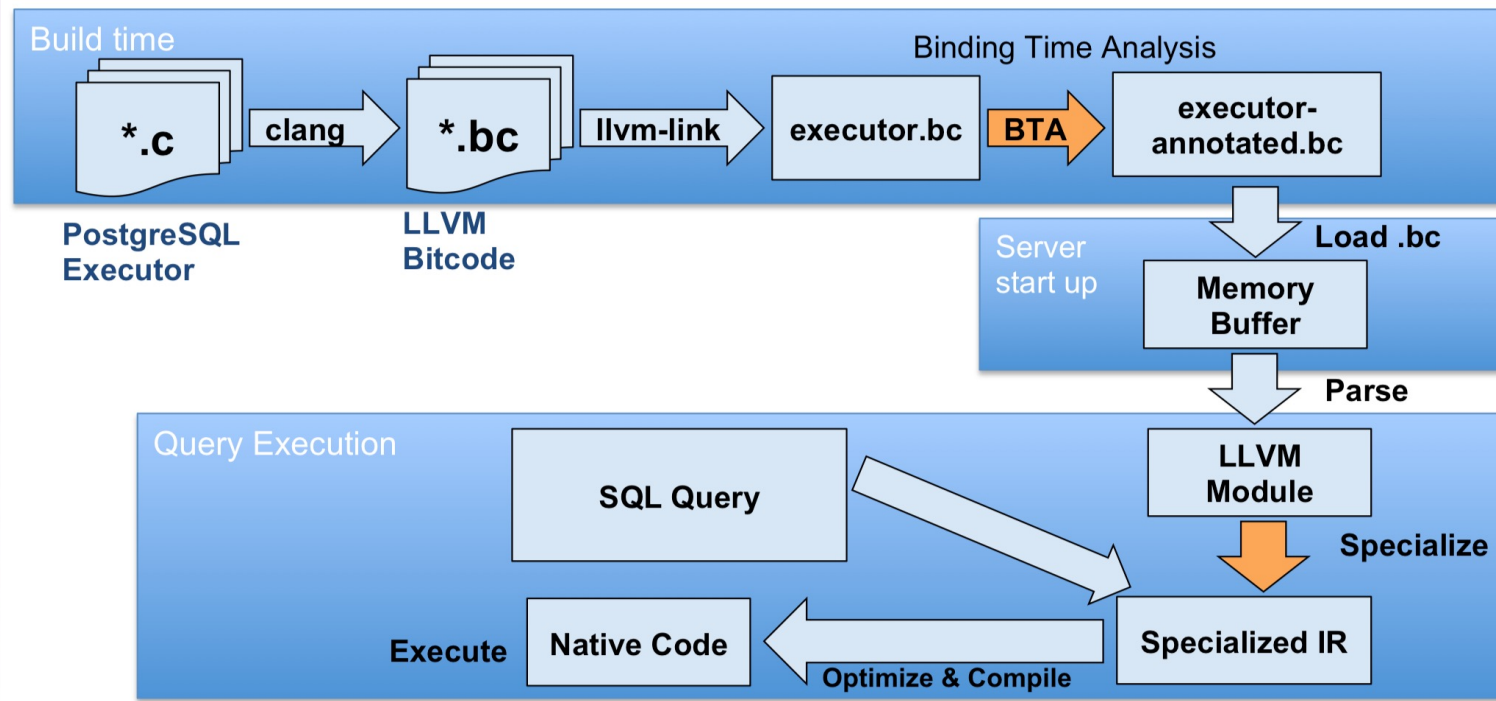
- 将查询计划直接转化为LLVM中间表示IR(intermediate representation), 在运行时即时编译为机器码; 在数据库进程内实时编译, 编译完成后立即执行。
- 编译代价抵: 直接调用LLVM, LLVM JIT可按需编译, 仅生成必要代码



源到源编译模型：HIQUE



- 1、利用代码模板将查询计划源到源编译为C/C++代码；
- 2、将代码编译为一个共享的对象；
- 3、与数据库管理系统链接以执行查询。





HIQUE模型示例

➤ 查询获取“某个学号以2021为开头的学生信息”

```
SELECT *  
FROM Student  
WHERE Student.Sno='2021'+?
```

传统模型:

```
for t in range(table.num_tuples):  
    tuple = get_tuple(table, t)  
    if eval(predicate, tuple, params):  
        emit(tuple)
```

- (1) 执行谓词表达式树，并拉取数值；
- (2) 在判断语句中计算偏移；
- (3) 在比较语句中进行必要的数值类型转换；
- (4) 返回True或False。

- (1) 获取目录中表的模式；
- (2) 根据元组大小计算偏移(offset)；
- (3) 返回元组指针。



HIQUE模型示例

- 查询获取 “某个学号以2021为开头的学生信息”

```
SELECT *  
FROM Student  
WHERE Student.Sno='2021'+?
```

HIQUE模型:

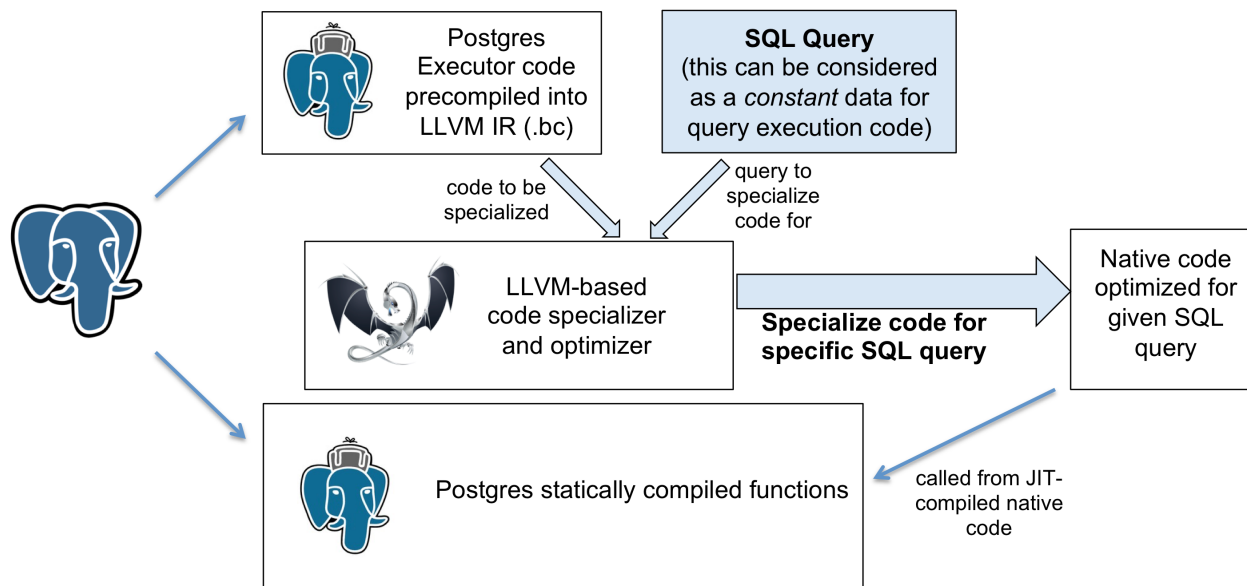
```
tuple_size = ###  
predicate_offset = ###  
parameter_value = ###  
  
for t in range(table.num_tuples):  
    tuple = table.data + t * tuple_size  
    Sno = (tuple + predicate_offset)  
    if (Sno == '2021' + parameter_value):  
        emit(tuple)
```

(###为预设参数)



LLM实时编译模型：HYPER

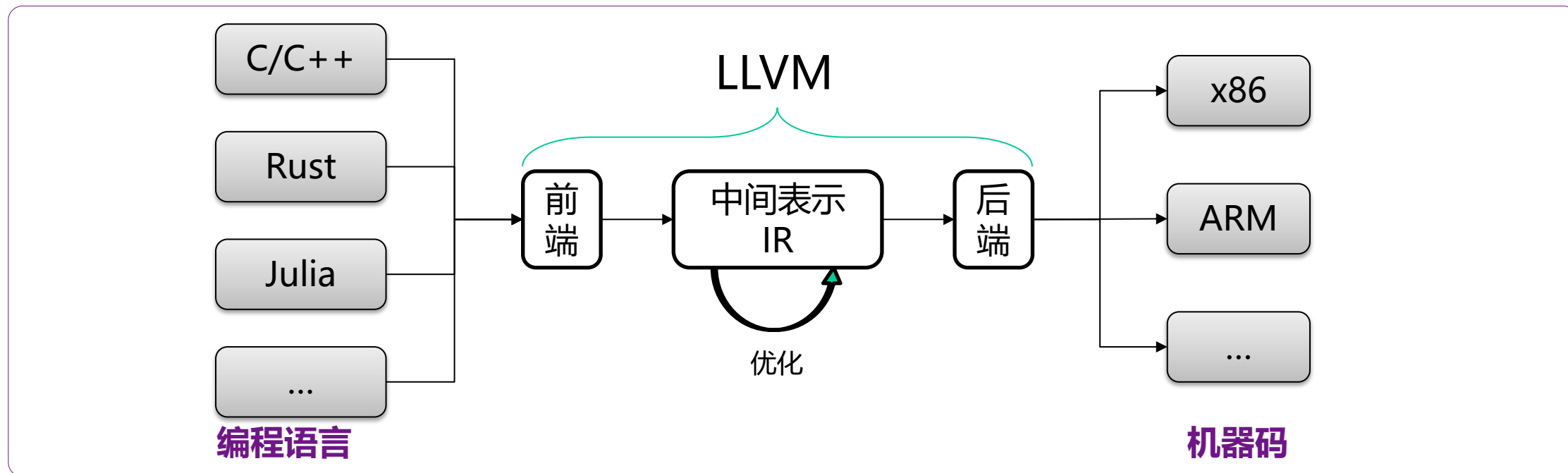
- HYPER是基于实时编译的代码生成模型；
- 并且是自下而上的、以数据为中心的推送式模型；
- 引入了流水线（pipeline）的概念。所谓流水线指的是查询计划的表达式树的一段，在这一段中，数据元组可以一直保留在寄存器中。





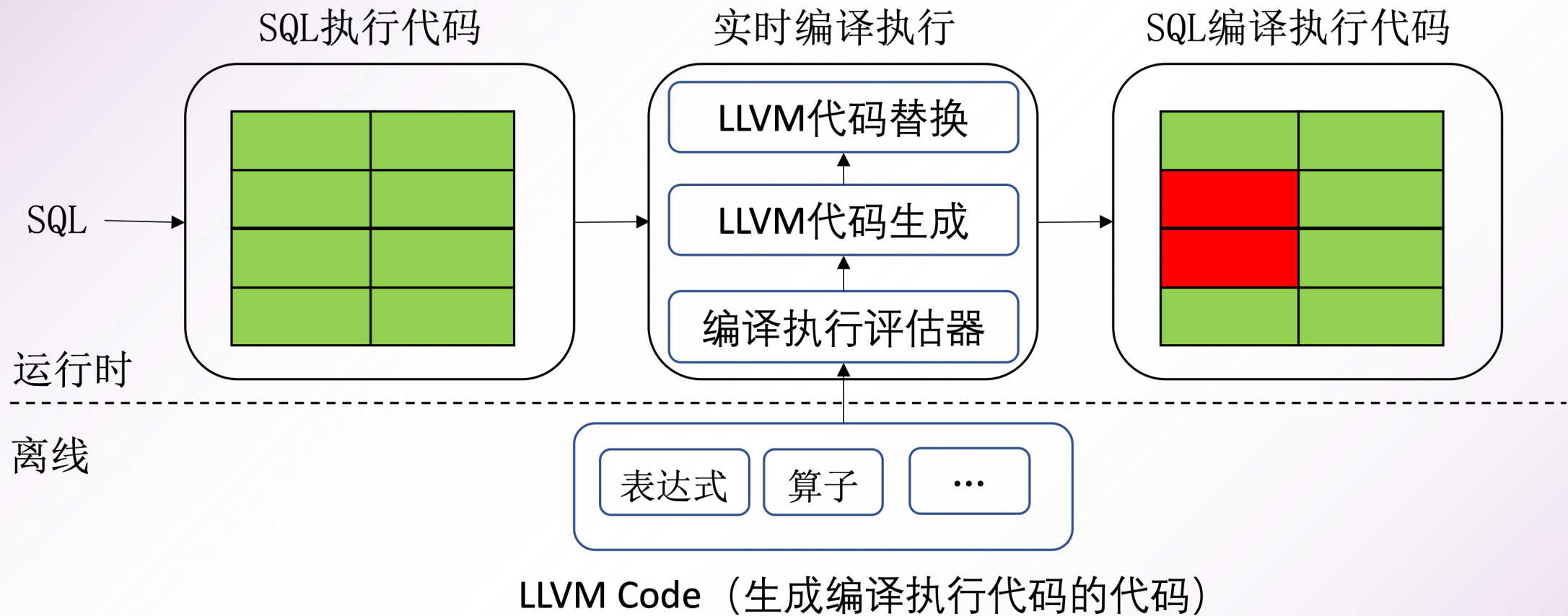
LLVM实时编译

- 模块化的架构，允许所有人对它进行改进；
- 后端允许将IR转换为不同CPU架构上的机器码；
- 前端允许将不同代码转换为IR；
- IR会不断地自我优化，直到达到最优解。





LLVM实时编译执行





LLM实时编译流程



□ 离线准备:

- 编译与依赖继承
 - 支持LLM链接库
 - JIT子系统初始化
 - LLVM引擎初始化、模块创建、代码生成入口
- 表达式/算子IR模板预定义
 - 编写SQL转换IR的算子库
 - 预定义所有算子的 IR 模板
 - 建立SQL类型和LLVM类型静态映射
 - 实现比较、算术、聚合、过滤的 IR 生成模板
 - 注册内核函数, 让 JIT 可以调用
- 添加JIT相关GUC参数

□ 在线执行:

- 执行器决策是否JIT, 调用编译入口
 - 基于代价判断
- 算子→IR 动态生成
 - 根据预定义模板生成IR
 - 表达式→IR的翻译器
 - 元组解构→IR的翻译器
- LLM优化与IR→机器码生成
 - 调用 LLVM 生成目标汇编与机器码
 - JIT执行重定位、符号解析
- JIT代码缓存
- 调用JIT函数执行查询



LLM实时编译的优势



1. 避免不必要的判断步骤

预先编译

```
void MyTuple(char* tuple)
{
    for (int i = 0; i < num_slots_; ++i)
    {
        char* slot = tuple + offsets_[i];
        switch(types_[i])
        {
            case INT:
                *slot = ParseInt();
                break;
            case FLOAT: ...
            case STRING: ...
            case BOOLEAN: ...
        }
    }
}
```

实时编译

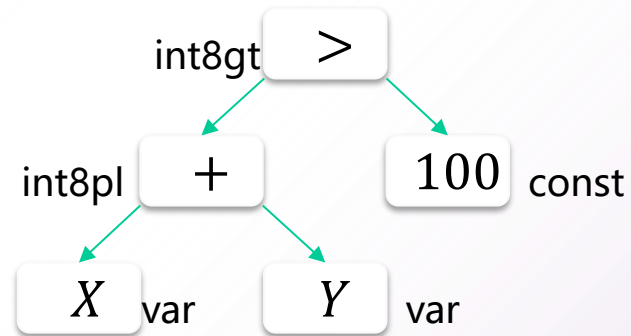
```
void MyTuple(char* tuple)
{
    *(tuple + 0) = ParseBoolean();
    *(tuple + 1) = ParseInt();
    *(tuple + 5) = ParseFloat();
}
```




LLM实时编译的优势

2. 优化表达式的计算

以表达式 “ $x + y > 100$ ” 为例：



预先编译

```
indirect call ExecEvalFunc()
  Left = indirect call ExecEvalFunc()
  X = indirect call ExecEvalVar()
  Y = indirect call ExecEvalVar()
  return int8pl(X, Y)
Right = indirect call ExecEvalConst(100)
return int8gt(Left, Right)
```

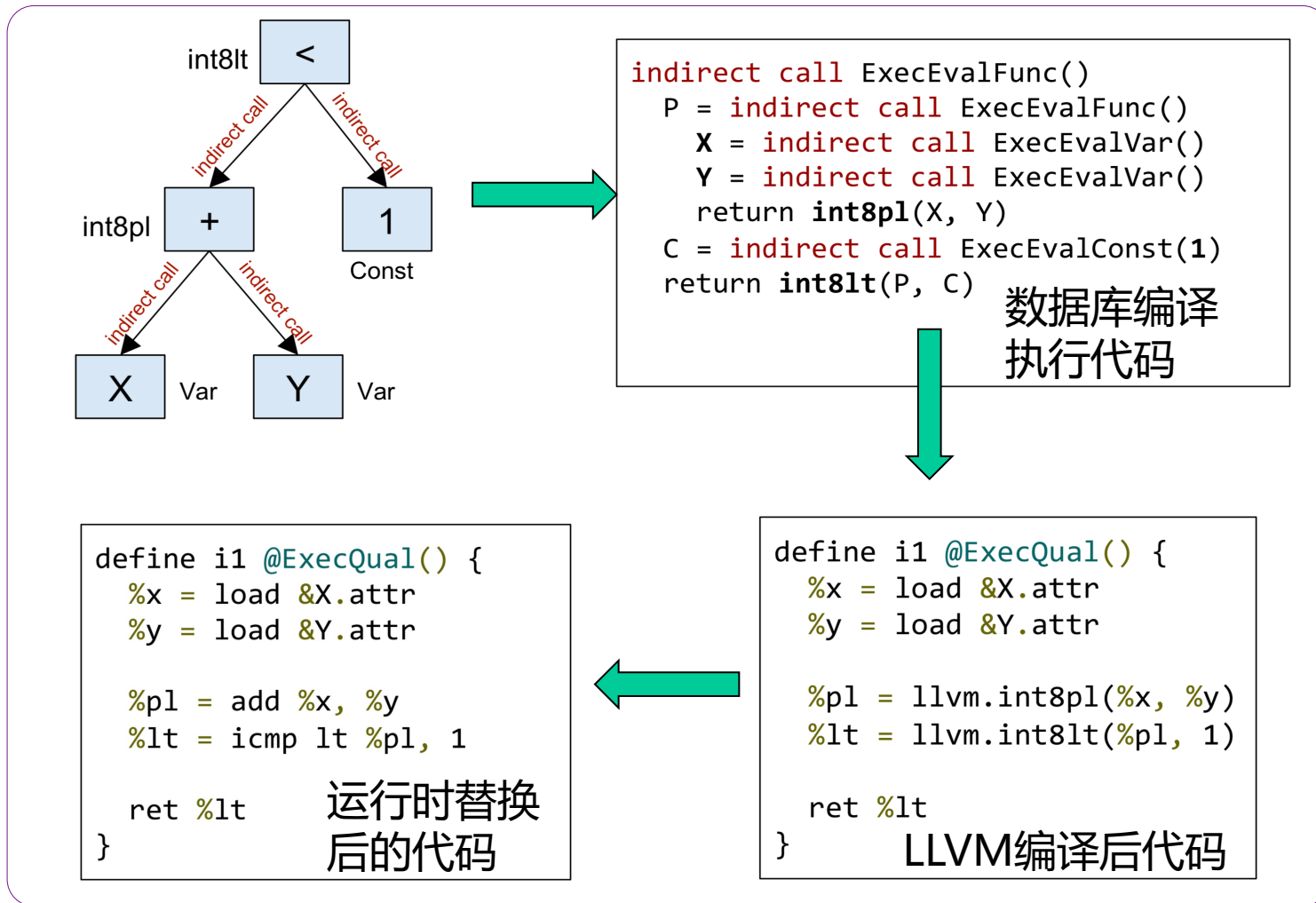
实时编译

```
define i1 @ExecQual()
{
  %x = load &X.attr
  %y = load &Y.attr
  %pl = llvm.int8pl(%x, %y)
  %gt = llvm.int8gt(%pl, 100)
  ret %gt
}
```



LLM实时编译的优势

2. 优化表达式的计算



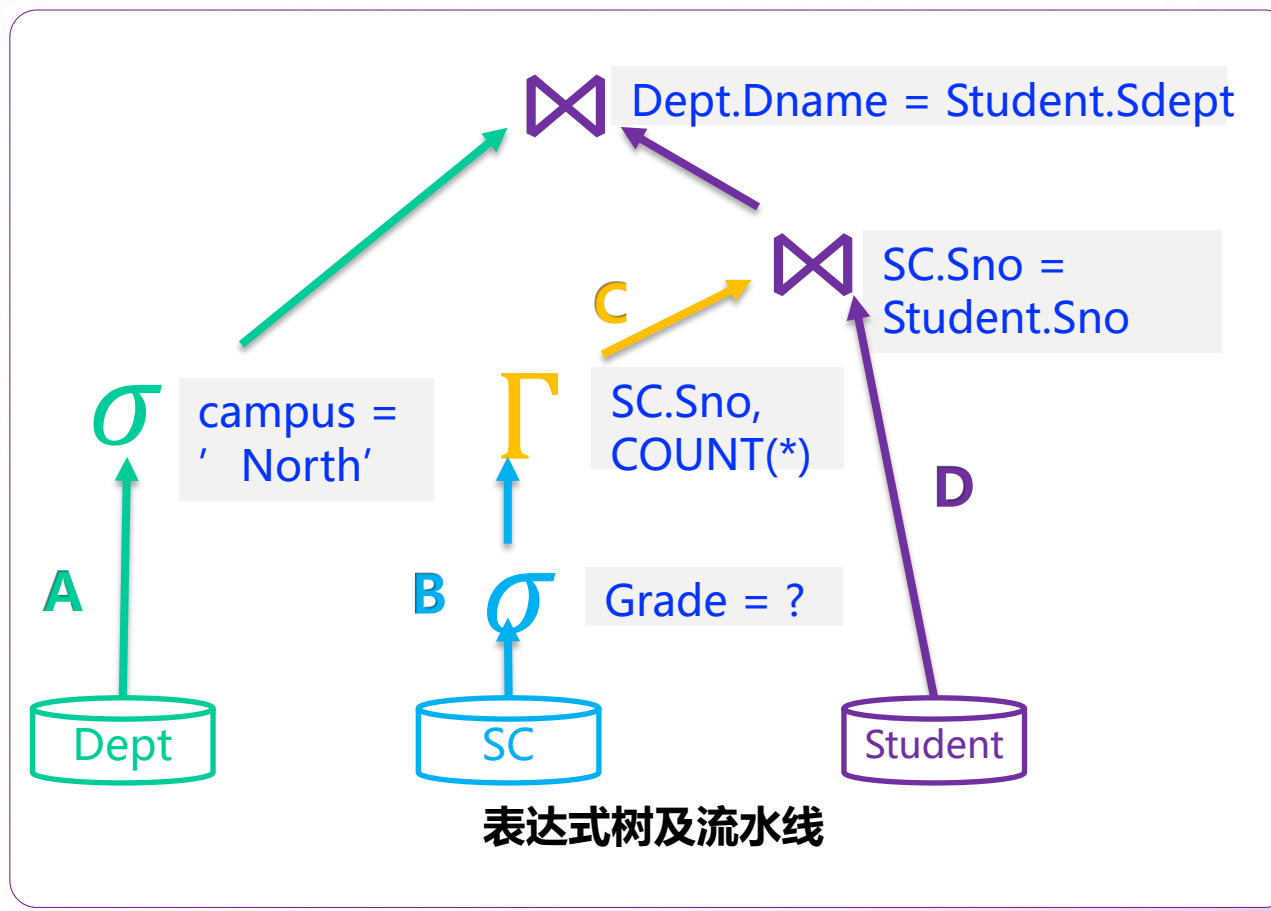


LLM实时编译的优势

3. 优化执行流程

在获取到查询计划之后，数据库就可以根据查询计划针对性地设计执行流程，使元组尽可能长的留存在寄存器中。

推送式模型



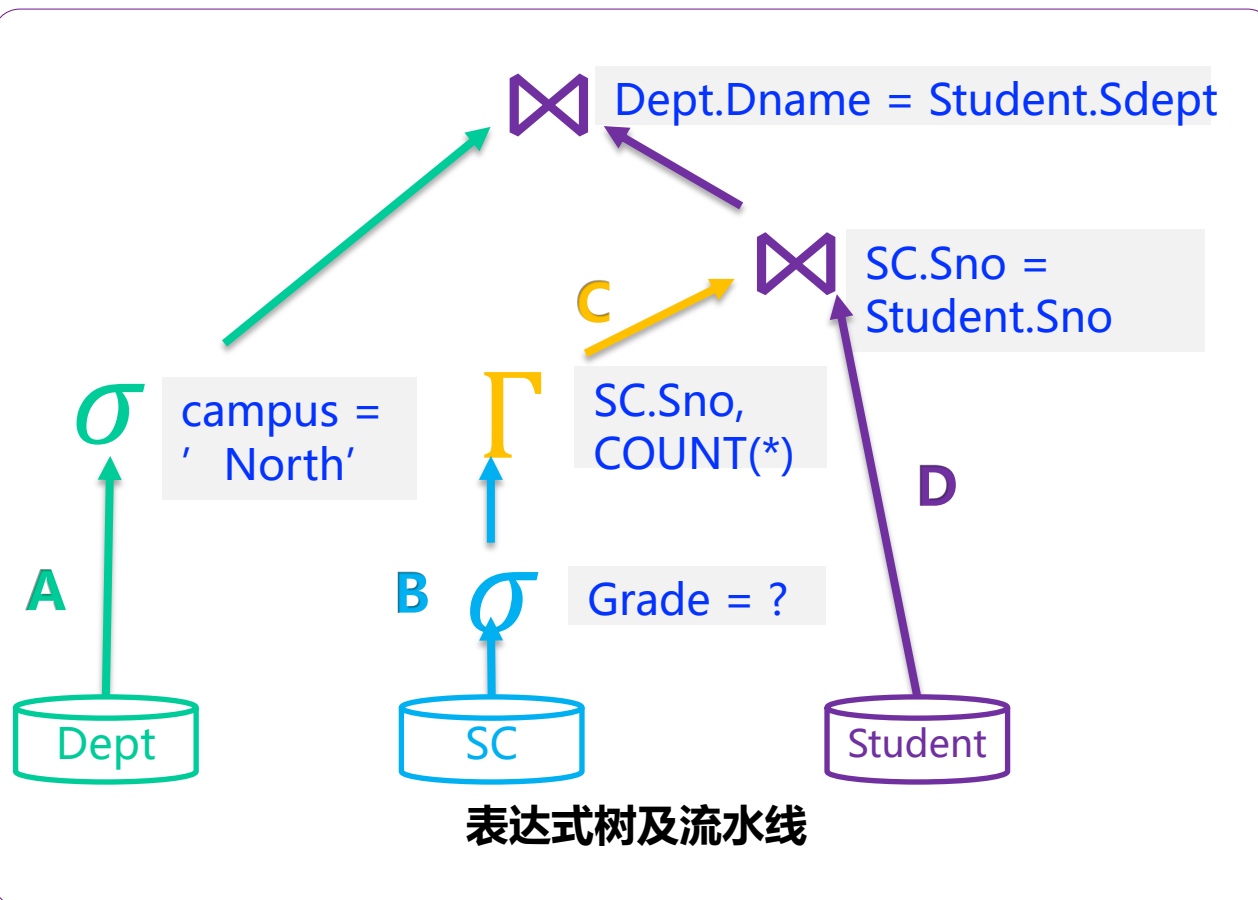


LLM实时编译： HYPER示例



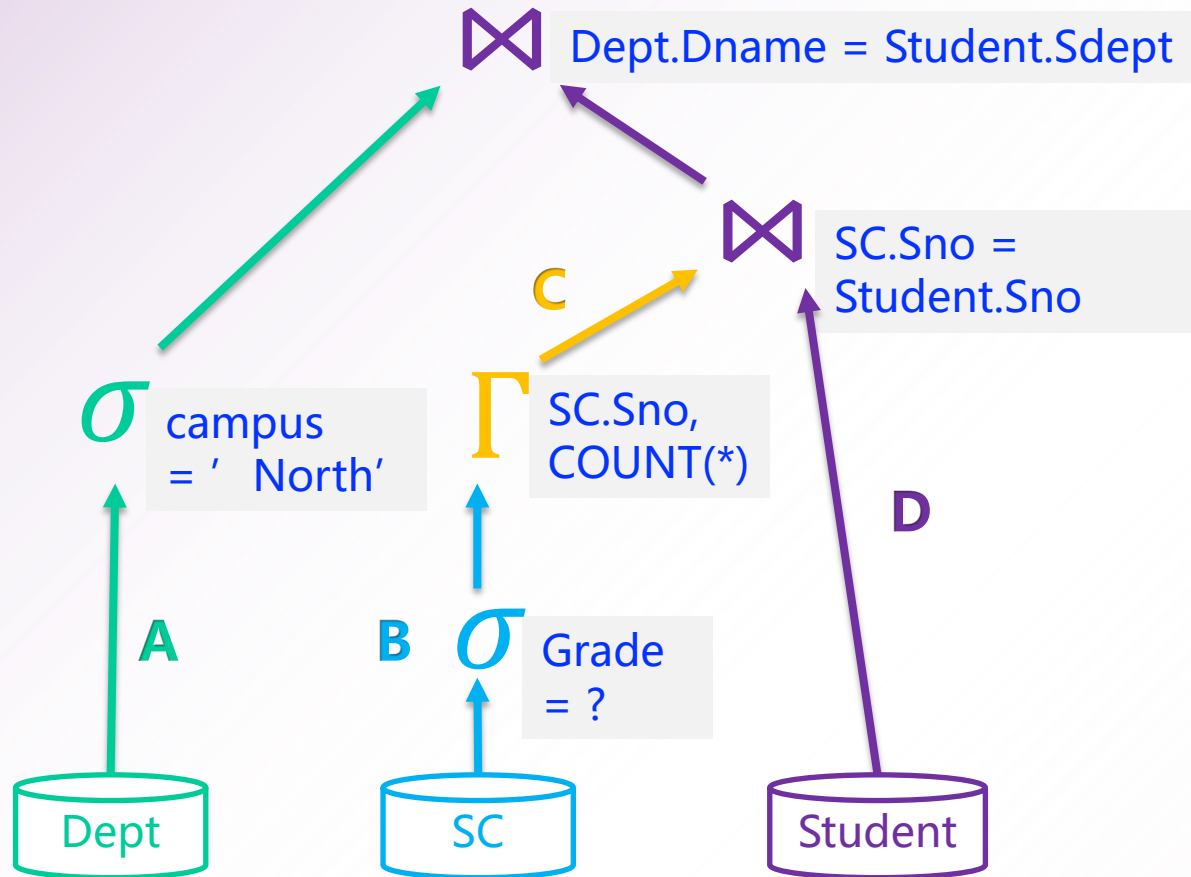
- 查询获取所有至少在一门课中获得某特定分数的北校区学生

```
SELECT *  
FROM Dept, Register,  
      (SELECT SC.Sno, COUNT(*)  
       FROM SC  
       WHERE SC.Grade = ?  
       GROUP BY SC.Sno) AS SC  
WHERE Dept.campus = 'North'  
AND Dept.Dname = Student.Sdept  
AND SC.Sno = Student.Sno
```





LLM实时编译： HYPER执行流程

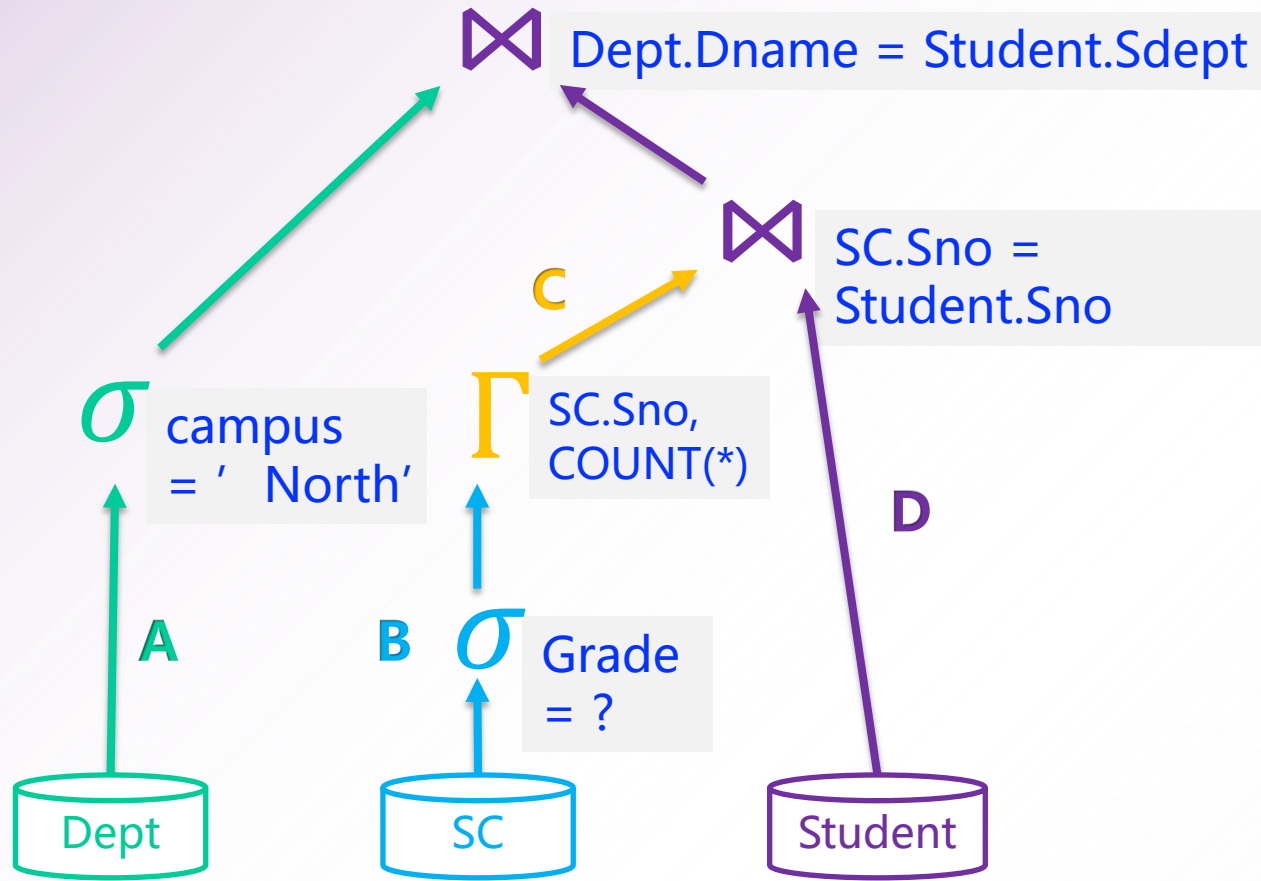


A:

```
for t in Dept:
    if Dept.campus = 'North':
        Materialize t in HashTable
Join  $\bowtie$  Dept.Dname = Student.Sdept
```




LLM实时编译： HYPER执行流程

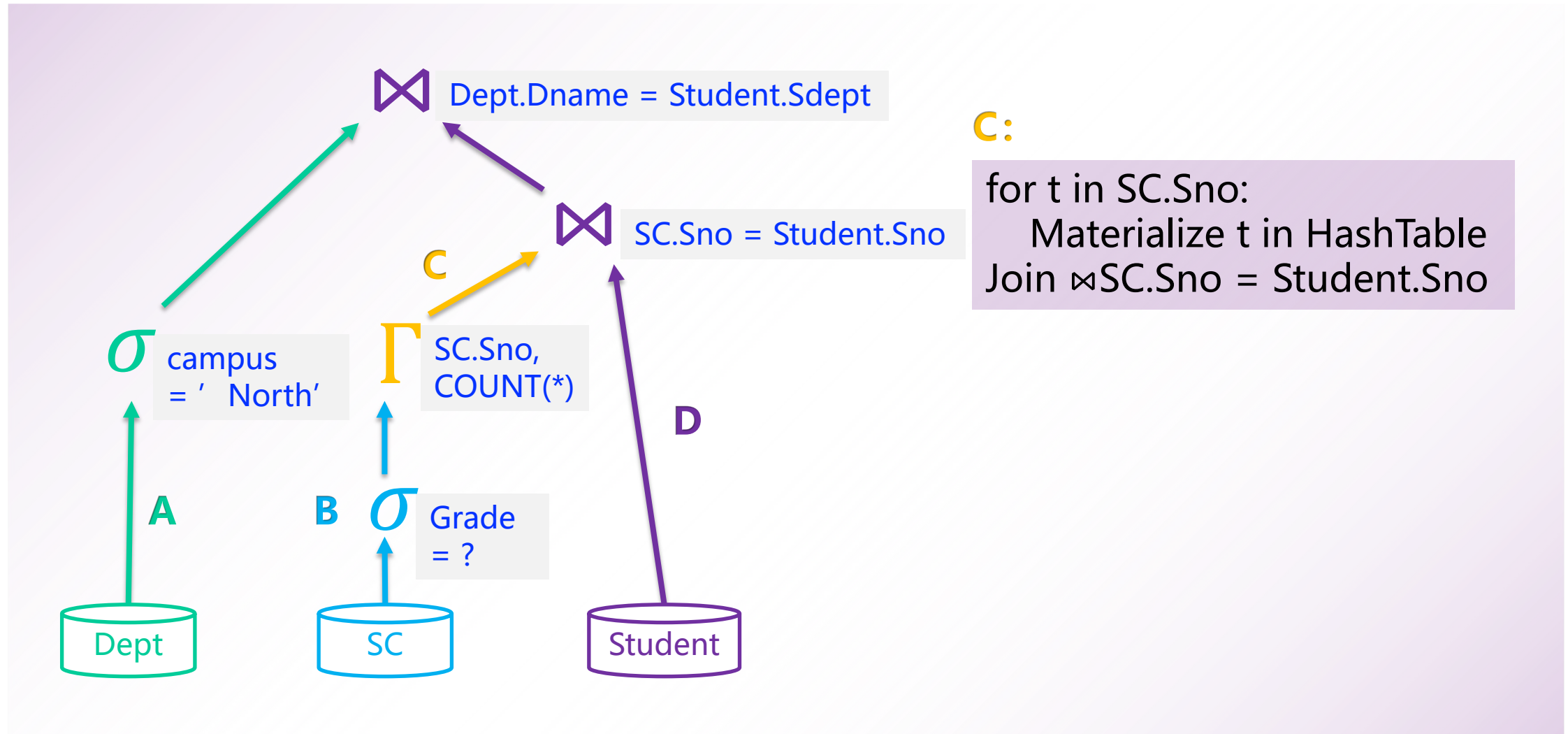


B:

```
for t in SC:
    if SC.Grade = <param>:
        Aggregate t in HashTable SC.Sno
```

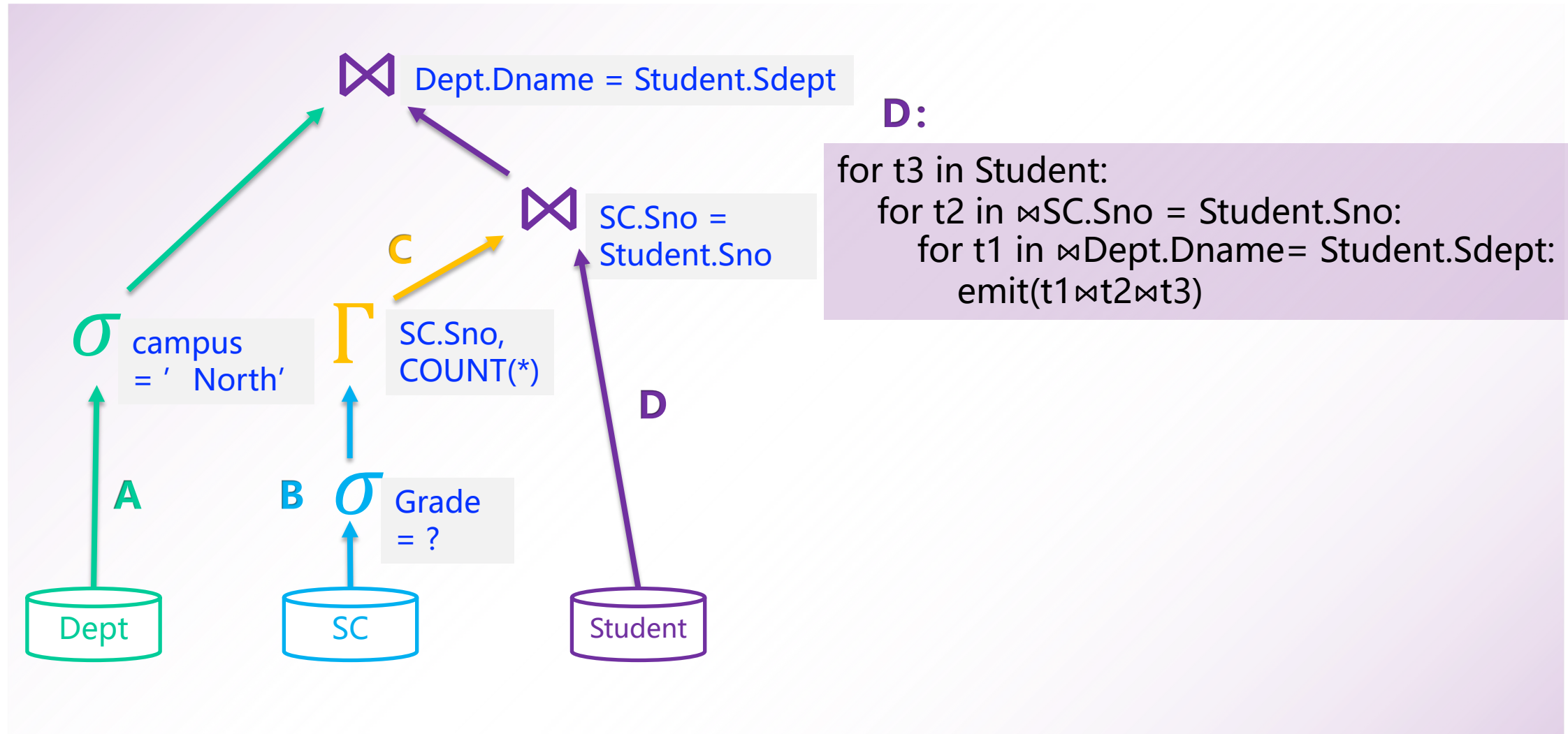


LLM实时编译： HYPER执行流程





LLM实时编译： HYPER执行流程





异步编译服务

- 为了降低编译本身的开销，可以异步编译提升性能
- 异步编译服务Compilation as a service
- 编译结果缓存



小结



- 本章介绍了数据库的执行模型
- 拉取式 vs 推送式
- 火山模型、物化模型、向量化模型
- 并行执行
 - MPP、SMP、SIMD
- 编译执行模型
 - AOT编译
 - JIT实时编译